

Tugas 3: Pembuatan Aplikasi Chat Web dengan Kriptografi Kunci-Simetri dan Kunci-Publik

II4021 - Kriptografi



Disusun oleh Kelompok 9:

Alma Felicia Vielrizki - 18223112

Aulia Azka Azzahra - 18223131

Sonya Putri Fadilah - 18223138

**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
FAKULTAS SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

DAFTAR ISI

DAFTAR ISI	2
DAFTAR TABEL	4
DAFTAR GAMBAR	5
I. PENDAHULUAN	8
1.1 Latar Belakang	8
1.2 Tujuan	9
1.3 Ruang lingkup aplikasi	9
II. Dasar Teori	10
2.1 JSON Web Token (JWT)	10
2.2 JWS dan ECDSA	11
2.3 Elliptic Curve Diffie-Hellman (ECDH)	12
2.4 HKDF	13
2.5 AES-256	14
2.6 Password hashing dan salt	15
2.7 Message Authentication Code (HMAC)	17
2.8 Docker	18
III. Perancangan Sistem	19
3.1 Arsitektur client-server	19
3.2 Skema basis data	20
3.3 Alur registrasi pengguna	21
3.4 Alur login dan autentikasi JWT	22
3.5 Alur daftar kontak	23
3.6 Alur pembentukan kunci komunikasi	25
3.7 Alur pengiriman pesan terenkripsi	27
3.8 Alur verifikasi MAC	29
3.9 Rancangan Docker	30
IV. Implementasi	32
4.1 Implementasi frontend	32
4.2 Implementasi API Client	34
4.2.1 userApi.js	35
4.2.2 messageApi.js	35
4.2.3 authApi.js	36
4.3 Implementasi backend API	37
4.4 Implementasi custom JWT library	39
4.4.1 Fungsi Sign (server/src/jwt-lib/sign.js)	39
4.4.2 Fungsi Verify (server/src/jwt-lib/verify.js)	39
4.4.3 Modul Pendukung	40
4.5 Implementasi database	41
4.5.1 Skema database	41
4.6 Implementasi enkripsi pesan	42
4.6.1 Enkripsi (aesEncrypt)	42
4.6.2 Dekripsi Pesan (aesDecrypt)	43
4.6.3 Import Kunci AES (importAesKey)	43

4.6.4 Integrasi Enkripsi pada Pengiriman Pesan	44
4.6.5 Integrasi Dekripsi pada Penerimaan Pesan	45
4.7 Implementasi MAC	45
4.8 Implementasi unit test JWT	46
4.9 Implementasi Docker	47
4.10 Penjelasan file penting pada repository	48
V. Pengujian dan Analisis	50
5.1 Registrasi dengan data valid	50
5.2 Registrasi dengan email terdaftar	52
5.3 Login dengan password benar	53
5.4 Login dengan password salah	54
5.5 JWT sign dan verify dengan public key benar	55
5.6 JWT verify dengan public key salah	56
5.7 JWT format tidak valid	57
5.8 JWT exp/nbf tidak sesuai	58
5.9 ECDH menghasilkan shared secret sama	59
5.10 HKDF menghasilkan kunci simetris	60
5.11 Pesan berhasil dienkripsi dan didekripsi	61
5.12 Dekripsi gagal dengan kunci/data salah	62
5.13 MAC valid	64
5.14 MAC tidak valid setelah pesan diubah	66
5.15 Pengujian Docker	68
VI. Kesimpulan	71
DAFTAR PUSTAKA	73
LAMPIRAN	74
A. Pranala repository	74
B. Pranala video demo	74
C. Pembagian tugas	74

DAFTAR TABEL

Tabel 2.1 Varian Algoritma ECDSA	12
Tabel 4.1 Halaman Utama Chat	32
Tabel 4.2 Fungsi pada userApi.js	35
Tabel 4.3 Fungsi pada messageApi.js	35
Tabel 4.4 Fungsi pada authApi.js	36
Tabel 4.5 Tabel users	41
Tabel 4.6 Tabel messages	42

DAFTAR GAMBAR

Gambar 3.1 Arsitektur client-server	19
Gambar 3.2 ERD database	20
Gambar 3.3 Alur registrasi pengguna	21
Gambar 3.4 Alur login dan autentikasi JWT	22
Gambar 3.5 Diagram Alur Daftar Kontak	23
Gambar 3.6 Diagram Alur Key Exchange	25
Gambar 3.7 Diagram Alur Pengiriman Pesan Terenkripsi	27
Gambar 3.8 Diagram Alur Verifikasi MAC	29
Gambar 3.9 Diagram Arsitektur Container Docker	30
Gambar 4.1 Antarmuka halaman Login	33
Gambar 4.2 Antarmuka halaman Register	33
Gambar 4.3 Antarmuka halaman Chat	34
Gambar 4.4 Antarmuka halaman Daftar Kontak	34
Gambar 4.5 Unit Tes JWT	47
Gambar 5.1 Unit Tes Registrasi Valid - Registrasi melalui Web	51
Gambar 5.2 Unit Tes Registrasi Valid - Hasil respon server	51
Gambar 5.3 Unit Tes Registrasi Valid - Hasil query database	51
Gambar 5.4 Unit Tes Registrasi Email terdaftar - Hasil respon server	52
Gambar 5.5 Unit Tes Registrasi Email terdaftar - Respon web	53
Gambar 5.6 Unit Tes Login - Hasil respon server	54
Gambar 5.7 Unit Tes Login Salah - Hasil respon server	55
Gambar 5.8 Unit Tes JWT - JWT sign dan verify dengan public key benar	56
Gambar 5.9 Unit Tes JWT - JWT verify dengan public key salah	57
Gambar 5.10 Unit Tes JWT - JWT format tidak valid	58
Gambar 5.11 Unit Tes JWT - JWT exp/nbf tidak sesuai	59
Gambar 5.12 Alice dan Bob Shared Secret pada output npm run test:crypto	60
Gambar 5.13 Kunci AES-256 yang dihasilkan HKDF di sisi User A dan B	60
Gambar 5.14 Tampilan pesan terkirim di sisi User A	61
Gambar 5.15 Data pesan terenkripsi (ciphertext) yang tersimpan di basis data/server	62
Gambar 5.16 Tampilan pesan di sisi User B (plaintext terdekripsi dengan benar)	62
Gambar 5.17 Ciphertext pesan sebelum dimodifikasi pada basis data	63
Gambar 5.18 Ciphertext pesan setelah dimodifikasi pada basis data	63
Gambar 5.19 Tampilan error akibat ciphertext yang dimodifikasi	64
Gambar 5.20 Tampilan pesan terkirim di sisi User A	65
Gambar 5.21 Tampilan pesan diterima dalam bentuk plaintext di sisi User B	65
Gambar 5.22 Data MAC sebelum dimodifikasi pada basis data	67
Gambar 5.23 Data MAC setelah dimodifikasi menjadi 'invalid-mac' pada basis data	67
Gambar 5.24 Tampilan "Invalid message authentication" di sisi User A	67
Gambar 5.25 Tampilan "Invalid message authentication" di sisi User B	68
Gambar 5.26 Output terminal proses build Docker Compose menunjukkan seluruh layer berhasil dibangun	69
Gambar 5.27 Output terminal menunjukkan ketiga service (db, server, client)	70
Gambar 5.28 Data users dan messages yang tersimpan pada PostgreSQL di dalam container	70

Gambar 5.29 Output terminal docker compose down -v menunjukkan seluruh container berhasil dihentikan

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Alma Felicia Vielrizki
18223112



Aulia Azka Azzahra
18223131



Sonya Putri Fadilah
18223138

I. PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi yang pesat telah mendorong meningkatnya penggunaan aplikasi chat sebagai sarana komunikasi sehari-hari. Kemudahan akses dan kecepatan pertukaran informasi menjadikan platform komunikasi digital semakin diminati, baik untuk keperluan personal maupun profesional. Namun, di balik kemudahan tersebut, terdapat risiko keamanan yang tidak dapat diabaikan, seperti penyadapan isi pesan, pencurian sesi autentikasi, hingga manipulasi pesan oleh pihak yang tidak bertanggung jawab.

Mekanisme autentikasi konvensional berbasis *username* dan *password* tidak cukup untuk menjamin keamanan komunikasi secara menyeluruh. Meskipun akses ke sistem dapat dibatasi, isi pesan yang ditransmisikan melalui jaringan tetap rentan terhadap intersepsi apabila tidak dilindungi. Oleh karena itu, diperlukan pendekatan keamanan yang lebih komprehensif, yang tidak hanya mengatur hak akses pengguna, tetapi juga memastikan kerahasiaan dan integritas pesan selama proses transmisi.

Untuk menjawab tantangan tersebut, kami mengembangkan sebuah aplikasi chat berbasis web yang mengintegrasikan beberapa mekanisme kriptografi modern. Autentikasi pengguna dilakukan menggunakan JSON Web Token (JWT) yang ditandatangani secara digital dengan algoritma *Elliptic Curve Digital Signature Algorithm* (ECDSA). Proses pembentukan kunci komunikasi antar pengguna menggunakan protokol *Elliptic Curve Diffie-Hellman* (ECDH) untuk menghasilkan *shared secret* yang kemudian diturunkan menjadi kunci simetris melalui HKDF. Selanjutnya, enkripsi pesan dilakukan menggunakan *Advanced Encryption Standard* (AES-256) sehingga server hanya berperan sebagai perantara yang tidak mengetahui isi plaintext pesan maupun kunci komunikasi yang digunakan.

Melalui tugas ini, sistem dirancang agar keamanan komunikasi sepenuhnya berada di sisi klien, sementara server hanya menyimpan dan meneruskan data terenkripsi. Dengan demikian, kami dapat memahami dan mengimplementasi sistem komunikasi aman berbasis kriptografi kunci publik dan simetris pada platform web.

1.2 Tujuan

Tujuan dari pembuatan program ini adalah sebagai berikut:

- Mengimplementasikan mekanisme autentikasi berbasis *JSON Web Token* (JWT) yang ditandatangani menggunakan algoritma ECDSA dengan *library* yang dibangun secara mandiri tanpa bergantung pada *library* JWT eksternal.
- Mengembangkan aplikasi chat berbasis web yang mendukung pendaftaran dan login pengguna dengan penyimpanan *password* secara aman menggunakan mekanisme *hashing* dan *salt*.
- Mengimplementasikan protokol *Elliptic Curve Diffie-Hellman* (ECDH) untuk pembentukan *shared secret* antar pengguna sebagai dasar pembentukan kunci komunikasi simetris.
- Menerapkan enkripsi dan dekripsi pesan menggunakan AES-256 sehingga server tidak dapat mengakses isi *plaintext* pesan yang dikirimkan antar pengguna.
- Membangun arsitektur sistem yang memisahkan tanggung jawab keamanan antara sisi klien dan sisi server, di mana seluruh operasi kriptografi sensitif dilakukan di sisi klien menggunakan Web Crypto API.
- Mengintegrasikan mekanisme *Message Authentication Code* (MAC) sebagai lapisan tambahan untuk memastikan integritas dan autentikasi pesan yang diterima.
- Menyediakan infrastruktur *deployment* berbasis Docker untuk memudahkan pengujian dan demonstrasi aplikasi secara konsisten di berbagai lingkungan.

1.3 Ruang lingkup aplikasi

Ruang lingkup dari program yang dikembangkan dalam tugas ini meliputi:

- Aplikasi dibangun berbasis web dengan arsitektur *client-server*, di mana sisi klien dikembangkan menggunakan React dan sisi server menggunakan Node.js dengan framework Express.
- Seluruh operasi kriptografi di sisi klien, termasuk pembangkitan pasangan kunci ECDH, proses key exchange, derivasi kunci menggunakan HKDF, serta enkripsi dan dekripsi pesan menggunakan AES-256, diimplementasikan menggunakan Web Crypto API.

- Fungsi *sign* dan *verify* JWT diimplementasikan secara mandiri sebagai *library* tersendiri tanpa menggunakan *library* JWT eksternal, dengan dukungan algoritma ES256, ES384, dan ES512 sesuai standar RFC 7519.
- Sistem mendukung alur komunikasi antara dua pengguna terdaftar, mulai dari registrasi, login, pertukaran kunci publik, hingga pengiriman dan penerimaan pesan terenkripsi.
- Server berperan sebagai perantara yang hanya menyimpan dan meneruskan data terenkripsi, tanpa memiliki akses terhadap *shared secret*, kunci komunikasi, maupun *plaintext* pesan.
- Penyimpanan data menggunakan basis data relasional PostgreSQL, mencakup data pengguna seperti email, *hashed password*, *salt*, *public key*, dan *encrypted private key*, serta riwayat pesan terenkripsi.
- Program menyediakan fitur verifikasi integritas pesan menggunakan *Message Authentication Code* (MAC) berbasis HMAC sebagai fitur tambahan.
- Konfigurasi deployment aplikasi disediakan dalam bentuk Docker dan Docker Compose untuk memudahkan eksekusi seluruh komponen sistem secara terpadu.

II. Dasar Teori

2.1 JSON Web Token (JWT)

JSON Web Token (JWT) adalah standar terbuka yang didefinisikan dalam RFC 7519 yang memungkinkan pertukaran informasi antar dua pihak secara ringkas dan aman dalam bentuk objek JSON. Informasi yang terkandung dalam JWT dapat dipercaya karena ditandatangani secara digital, sehingga integritasnya dapat diverifikasi oleh penerima.

JWT terdiri dari tiga bagian yang dipisahkan oleh karakter titik (.), dengan format:

<code>base64url(header).base64url(payload).base64url(signature)</code>
--

Header berisi metadata token, yaitu jenis token (typ) dan algoritma tanda tangan yang digunakan (alg). Contoh:

```
{
  "alg": "ES256",
  "typ": "JWT"
}
```

Payload berisi klaim (*claims*), yaitu pernyataan mengenai suatu entitas (biasanya pengguna) beserta data tambahan. Klaim terbagi menjadi tiga jenis:

- *Registered claims*: klaim standar yang terdefinisi dalam RFC 7519, seperti iss (*issuer*), sub (*subject*), aud (*audience*), exp (*expiration time*), nbf (*not before*), iat (*issued at*), dan jti (JWT ID).
- *Public claims*: klaim yang dapat didefinisikan bebas namun harus terdaftar untuk menghindari konflik.
- *Private claims*: klaim khusus yang disepakati antar pihak yang berkomunikasi.

Signature dihasilkan dengan menandatangani gabungan *header* dan *payload* yang sudah di-*encode* menggunakan algoritma yang dispesifikasikan di header. *Signature* inilah yang menjamin bahwa token tidak dimanipulasi selama transmisi.

Pada tugas ini, JWT digunakan sebagai mekanisme autentikasi sesi pengguna. Server menerbitkan JWT saat login berhasil, dan klien menyertakan JWT tersebut pada setiap permintaan berikutnya untuk membuktikan identitasnya.

2.2 JWS dan ECDSA

JSON Web Signature (JWS) adalah salah satu bentuk implementasi JWT di mana payload ditandatangani secara digital. JWS didefinisikan dalam RFC 7515 dan memastikan bahwa isi token tidak dapat dimodifikasi tanpa terdeteksi, karena setiap perubahan akan membuat signature menjadi tidak valid.

Pada tugas ini, JWT diimplementasikan dalam format JWS dan ditandatangani menggunakan ECDSA (*Elliptic Curve Digital Signature Algorithm*). ECDSA adalah algoritma tanda tangan digital berbasis kurva eliptik yang menawarkan keamanan setara dengan RSA namun dengan ukuran kunci yang jauh lebih kecil, sehingga lebih efisien untuk digunakan dalam token.

Proses sign JWT dengan ECDSA berjalan sebagai berikut:

1. *Header* dan *payload* masing-masing di-*encode* ke Base64URL.
2. String `base64url(header).base64url(payload)` digunakan sebagai *signing input*.
3. ECDSA menghitung tanda tangan dari signing input menggunakan *private key*.
4. Signature di-encode ke Base64URL dan digabungkan menjadi token final.

Proses verify JWT dengan ECDSA berjalan sebagai berikut:

1. Token dipisahkan menjadi tiga segmen.
2. Signature didekode dari Base64URL.
3. ECDSA memverifikasi bahwa *signature* sesuai dengan signing input menggunakan *public key*.
4. Jika valid, klaim-klaim dalam payload divalidasi (exp, nbf, iss, sub, aud, jti).

Library JWT kustom pada tugas ini mendukung tiga varian algoritma ECDSA:

Tabel 2.1 Varian Algoritma ECDSA

Algoritma	Kurva	Hash
ES256	P-256 (prime256v1)	SHA-256
ES384	P-384 (secp384r1)	SHA-384
ES512	P-521 (secp521r1)	SHA-512

Signature ECDSA yang dihasilkan menggunakan format IEEE P1363 (R // S), sesuai dengan standar JWT, bukan format DER yang merupakan *default* Node.js.

2.3 Elliptic Curve Diffie-Hellman (ECDH)

Elliptic Curve Diffie-Hellman (ECDH) adalah protokol pertukaran kunci yang memungkinkan dua pihak menghasilkan *shared secret* yang sama melalui saluran komunikasi yang tidak aman, tanpa perlu saling mengirimkan kunci rahasia secara langsung.

ECDH bekerja berdasarkan sifat matematis kurva eliptik. Misalkan dua pengguna, Alice dan Bob, masing-masing memiliki pasangan kunci:

- Alice: private key a , public key $A = a \cdot G$
- Bob: private key b , public key $B = b \cdot G$

di mana G adalah titik generator pada kurva. *Shared secret* dihitung sebagai:

- Alice menghitung: $S = a \cdot B = a \cdot b \cdot G$
- Bob menghitung: $S = b \cdot A = b \cdot a \cdot G$

Kedua hasil identik karena perkalian titik pada kurva eliptik bersifat komutatif. Penyerang yang hanya mengetahui public key A dan B tidak dapat menghitung S tanpa mengetahui salah satu *private key*. Ini dikenal sebagai **Elliptic Curve Discrete Logarithm Problem (ECDLP)**.

Pada tugas ini, ECDH menggunakan kurva P-256 (prime256v1) melalui Web Crypto API. Setiap pengguna memiliki pasangan kunci ECDH yang dibangkitkan saat registrasi. Ketika dua pengguna mulai berkomunikasi, masing-masing pihak mengambil *public key* lawan dari server, lalu menghitung *shared secret* secara lokal di browser. Server sama sekali tidak mengetahui nilai *shared secret* ini karena *private key* tidak pernah dikirim ke server dalam bentuk *plaintext*.

2.4 HKDF

HKDF (HMAC-based Key Derivation Function) adalah fungsi derivasi kunci yang didefinisikan dalam RFC 5869. HKDF dirancang untuk mengambil bahan kunci awal (*Input Keying Material*) yang berpotensi lemah atau tidak seragam, kemudian menghasilkan satu atau lebih kunci kriptografi yang kuat dan terdistribusi secara seragam [1]. HKDF terdiri dari dua tahap, yaitu *Extract* dan *Expand*.

Tahap pertama adalah *extract*, yang menggunakan HMAC untuk mengubah IKM menjadi nilai pseudorandom key (PRK) berukuran tetap, dengan bantuan sebuah nilai *salt* yang bersifat opsional namun direkomendasikan. Tahap kedua adalah *expand*, yang memperluas PRK menjadi kunci keluaran sepanjang yang dibutuhkan menggunakan parameter info sebagai konteks dan pemisahan domain.

Secara formal, HKDF didefinisikan sebagai berikut:

$\begin{aligned} \text{HKDF-Extract}(\text{salt}, \text{IKM}) &= \text{HMAC-Hash}(\text{salt}, \text{IKM}) = \text{PRK} \\ \text{HKDF-Expand}(\text{PRK}, \text{info}, L) &= T(1) \parallel T(2) \parallel \dots T(N) \end{aligned}$
--

di mana L adalah panjang kunci keluaran yang diinginkan dalam byte, dan setiap blok $T(i)$ dihitung secara iteratif dari blok sebelumnya dengan menggunakan HMAC [1].

Dalam konteks sistem chat ini, HKDF digunakan setelah proses ECDH untuk mengubah *shared secret* yang dihasilkan menjadi kunci AES-256 yang siap digunakan. Alasan HKDF diperlukan adalah karena output ECDH berupa koordinat titik pada kurva eliptik tidak memiliki distribusi bit yang seragam dan tidak memiliki panjang yang pas untuk langsung digunakan sebagai kunci AES. HKDF menyelesaikan masalah ini dengan menghasilkan kunci yang terstandarisasi, kuat secara kriptografis, dan dapat direproduksi oleh kedua pihak secara independen [2].

2.5 AES-256

Advanced Encryption Standard (AES) adalah algoritma enkripsi simetris blok yang ditetapkan oleh *National Institute of Standards and Technology* (NIST) pada tahun 2001 melalui FIPS PUB 197, menggantikan standar DES yang sudah usang. AES diadopsi secara luas di seluruh dunia sebagai standar enkripsi karena keamanannya yang terbukti dan efisiensinya pada berbagai platform perangkat keras maupun perangkat lunak [3].

AES adalah cipher blok yang beroperasi pada blok data berukuran 128 bit (16 byte). AES mendukung tiga ukuran kunci: 128 bit, 192 bit, dan 256 bit. Varian AES-256 menggunakan kunci sepanjang 256 bit dan menjalankan 14 putaran (*rounds*) transformasi, menjadikannya varian AES dengan tingkat keamanan tertinggi.

Setiap putaran AES terdiri dari empat operasi utama: SubBytes (substitusi byte nonlinier menggunakan S-box), ShiftRows (pergeseran baris pada state matrix), MixColumns (pencampuran kolom menggunakan operasi pada $GF(2^8)$), dan AddRoundKey (XOR state dengan subkunci putaran).

Dalam implementasi sistem ini, AES-256 digunakan dalam mode **Galois/Counter Mode (GCM)**. AES-GCM merupakan mode operasi autentikasi-enkripsi (AEAD) yang menggabungkan enkripsi dengan verifikasi integritas dalam satu operasi. Mode ini menghasilkan ciphertext beserta sebuah authentication tag (umumnya 128 bit) yang dapat digunakan untuk memverifikasi bahwa data tidak dimanipulasi selama transmisi [4]. Mode ini dipilih karena didukung secara *native* oleh Web Crypto API, tidak memerlukan *padding* seperti mode CBC sehingga menghindari serangan *padding oracle*, dan direkomendasikan oleh NIST sebagai mode operasi modern [4].

AES-GCM memerlukan sebuah Initialization Vector (IV) atau *nonce* yang unik untuk setiap operasi enkripsi dengan kunci yang sama. Penggunaan ulang IV dengan kunci yang sama merupakan kelemahan kritis yang dapat membongkar kunci enkripsi. Oleh karena itu, IV dibangkitkan secara acak menggunakan generator bilangan acak kriptografis (CSPRNG) untuk setiap pesan yang dienkripsi [4].

2.6 Password hashing dan salt

Password merupakan salah satu informasi paling sensitif yang disimpan oleh sebuah sistem. Menyimpan *password* dalam bentuk *plaintext* merupakan praktik yang sangat berbahaya karena apabila basis data berhasil diakses oleh pihak yang tidak berwenang, seluruh *password* pengguna akan langsung terekspos. Untuk mengatasi hal ini, digunakan mekanisme *password* hashing yang mengubah *password* menjadi representasi tetap yang tidak dapat dikembalikan ke bentuk aslinya.

Hashing berbeda dengan enkripsi. Enkripsi bersifat dua arah, artinya data yang dienkripsi dapat didekripsi kembali menggunakan kunci yang sesuai. Sebaliknya, hashing bersifat satu arah, artinya hasil hash tidak dapat dikembalikan ke nilai aslinya secara komputasional. Ketika pengguna melakukan *login*, sistem tidak mendekripsi hash yang tersimpan, melainkan menghash kembali *password* yang diinputkan dan membandingkan hasilnya dengan hash yang tersimpan di basis data.

Fungsi hash kriptografis memiliki beberapa sifat penting, yaitu:

- *Pre-image resistance*: Diberikan nilai hash h , secara komputasional tidak mungkin menemukan input m sedemikian sehingga $\text{hash}(m) = h$.
- *Second pre-image resistance*: Diberikan input m_1 , secara komputasional tidak mungkin menemukan $m_2 \neq m_1$ sedemikian sehingga $\text{hash}(m_1) = \text{hash}(m_2)$.
- *Collision resistance*: Secara komputasional tidak mungkin menemukan dua input berbeda m_1 dan m_2 yang menghasilkan nilai hash yang sama.

Namun, penggunaan fungsi hash kriptografis biasa seperti MD5 atau SHA-256 secara langsung untuk menyimpan password tidak disarankan. Hal ini disebabkan oleh beberapa kelemahan, antara lain kerentanan terhadap *rainbow table attack* dan *dictionary attack*. *Rainbow table* adalah tabel pra komputasi yang memetakan nilai hash ke plaintext yang menghasilkannya, sehingga penyerang dapat dengan cepat menemukan password asli hanya dengan mencari nilai hash di tabel tersebut.

Untuk mengatasi kelemahan tersebut, digunakan teknik salt, yaitu nilai acak unik yang ditambahkan ke setiap password sebelum proses hashing dilakukan. Dengan adanya salt, dua pengguna yang memiliki password identik akan menghasilkan nilai hash yang berbeda karena salt yang digunakan berbeda. Hal ini secara efektif menggugurkan penggunaan *rainbow table* karena penyerang harus membangun tabel baru untuk setiap kombinasi salt yang berbeda.

Salt tidak perlu dirahasiakan dan dapat disimpan bersama hash di basis data. Fungsi salt bukan untuk menambah kerahasiaan, melainkan untuk memastikan keunikan setiap hash meskipun password yang digunakan sama.

Pada implementasi ini, digunakan algoritma bcrypt untuk proses *password hashing*. Bcrypt merupakan fungsi hash yang dirancang khusus untuk keperluan penyimpanan password dan memiliki beberapa keunggulan dibandingkan fungsi hash kriptografis biasa.

Bcrypt secara internal menangani pembangkitan dan penyimpanan salt secara otomatis, sehingga setiap hash yang dihasilkan sudah mengandung informasi salt di dalamnya. Format output bcrypt adalah sebagai berikut:

Keunggulan utama bcrypt adalah adanya *cost factor*, yaitu parameter yang menentukan jumlah iterasi komputasi yang diperlukan untuk menghasilkan satu nilai hash. Pada implementasi ini digunakan *cost factor* 12, yang berarti algoritma melakukan $2^{12} = 4.096$ iterasi sebelum menghasilkan hash akhir. Semakin tinggi nilai *cost factor*, semakin lambat proses hashing dan semakin sulit bagi penyerang untuk melakukan serangan brute force. Keunggulan lain dari bcrypt adalah sifatnya yang adaptif, artinya *cost factor* dapat ditingkatkan seiring dengan peningkatan kemampuan komputasi di masa depan tanpa mengubah algoritma yang digunakan.

Selain salt untuk keperluan bcrypt, sistem juga membangkitkan KDF salt (*Key Derivation Function salt*) yang terpisah. KDF salt ini digunakan oleh sisi klien untuk menurunkan kunci AES dari password pengguna melalui proses *key derivation*, yang kemudian digunakan untuk mengenkripsi private key ECDH sebelum dikirim ke server. KDF salt disimpan terpisah di basis data dan dikembalikan ke klien saat proses login agar klien dapat memulihkan private key yang terenkripsi menggunakan password yang sama. Pemisahan antara salt untuk bcrypt dan salt untuk KDF ini

penting untuk memastikan kedua proses kriptografis berjalan secara independen dan tidak saling mempengaruhi.

Implementasi password hashing pada sistem ini terdapat pada file ``server/src/utlis/passwordHash.js``. Proses hashing dilakukan menggunakan library ``bcrypt`` dengan cost factor 12, sedangkan KDF salt dibangkitkan menggunakan ``crypto.randomBytes(32)`` untuk menghasilkan 32 byte nilai acak yang kriptografis. Pada saat login, verifikasi dilakukan menggunakan fungsi ``bcrypt.compare()`` yang secara otomatis mengekstrak salt dari hash yang tersimpan dan membandingkan hasilnya dengan password yang diinputkan pengguna.

2.7 Message Authentication Code (HMAC)

Message Authentication Code (MAC) adalah nilai kriptografi singkat yang dihitung dari pesan dan kunci rahasia, digunakan untuk memverifikasi integritas dan autentisitas pesan. Berbeda dengan tanda tangan digital, MAC menggunakan kunci simetris, hanya pihak yang memegang kunci yang sama yang dapat menghasilkan dan memverifikasi MAC.

Pada tugas ini digunakan HMAC (Hash-based MAC) dengan algoritma HMAC-SHA-256, yang didefinisikan dalam RFC 2104. HMAC menghitung MAC sebagai:

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$$

di mana H adalah fungsi hash (SHA-256), K adalah kunci, m adalah pesan, dan opad/ipad adalah konstanta *padding*.

Alur pengiriman pesan dengan MAC pada sistem ini:

1. Pengirim mengenkripsi pesan menggunakan AES-256-GCM, menghasilkan *ciphertext*.
2. Pengirim menghitung HMAC-SHA-256(*hmacKey*, *ciphertext*), menghasilkan *mac*.
3. Pengirim mengirim { *ciphertext*, *iv*, *mac* } ke server.

Alur penerimaan pesan:

1. Penerima memverifikasi MAC: `verify(hmacKey, ciphertext, mac)`.
2. Jika MAC tidak valid, pesan ditandai sebagai tidak valid, proses berhenti.
3. Jika MAC valid, ciphertext didekripsi dan pesan ditampilkan.

Kunci HMAC (**hmacKey**) diturunkan secara terpisah dari shared secret ECDH menggunakan HKDF dengan nilai info yang berbeda dari kunci AES ("chat-hmac-key-v1" vs "chat-aes-key-v1"), sehingga kedua kunci tidak saling bergantung meskipun berasal dari material yang sama.

Penggunaan `crypto.subtle.verify()` pada Web Crypto API untuk verifikasi MAC memastikan perbandingan dilakukan secara *constant-time*, sehingga aman terhadap *timing attack*.

2.8 Docker

Docker adalah platform virtualisasi berbasis container yang memungkinkan pengembang mengemas aplikasi beserta seluruh dependensinya ke dalam unit yang dapat dijalankan secara konsisten di berbagai lingkungan. Berbeda dengan virtual machine yang memvirtualisasi seluruh sistem operasi, container Docker berbagi kernel OS *host* sehingga lebih ringan dan cepat.

Komponen utama Docker yang digunakan pada tugas ini:

Dockerfile adalah file teks berisi instruksi untuk membangun image Docker suatu service. Setiap instruksi (seperti FROM, COPY, RUN, CMD) membentuk *layer* yang di-*cache*, sehingga proses *build* menjadi efisien.

Docker Compose adalah alat untuk mendefinisikan dan menjalankan aplikasi *multi-container*. Konfigurasi seluruh service (client, server, database) ditulis dalam satu file `docker-compose.yml`, sehingga seluruh aplikasi dapat dijalankan dengan satu perintah:

```
docker compose up
```

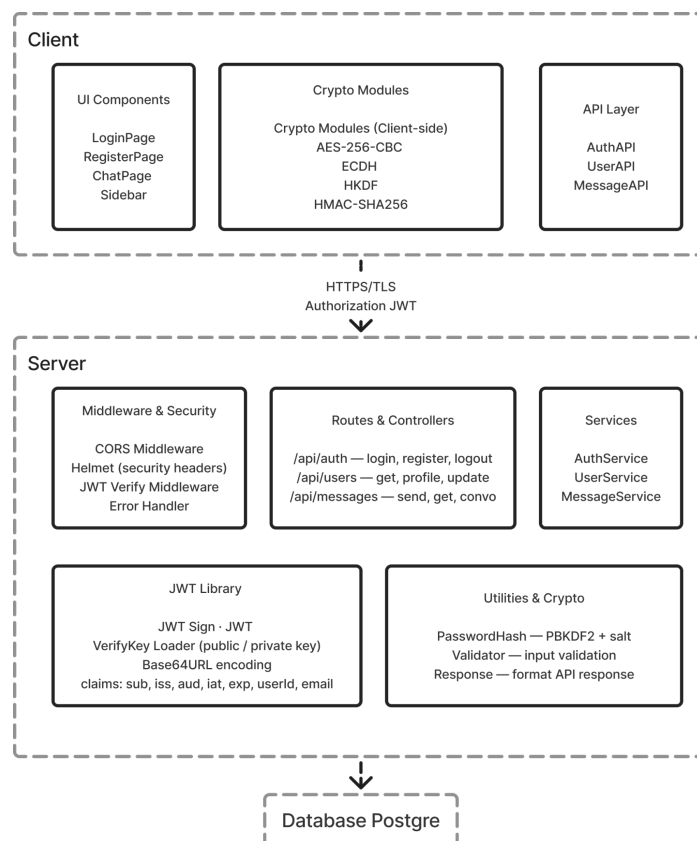
Pada tugas ini, aplikasi dikemas menjadi tiga container:

- client: aplikasi frontend React yang di-serve oleh Vite development server atau Nginx.
- server: REST API berbasis Node.js/Express.
- db: database PostgreSQL untuk menyimpan data pengguna dan pesan terenkripsi.

Docker Compose mengatur jaringan antar *container* sehingga *service* dapat saling berkomunikasi menggunakan nama *service* sebagai *hostname* (misalnya server mengakses database melalui host db), tanpa perlu konfigurasi jaringan manual.

III. Perancangan Sistem

3.1 Arsitektur client-server



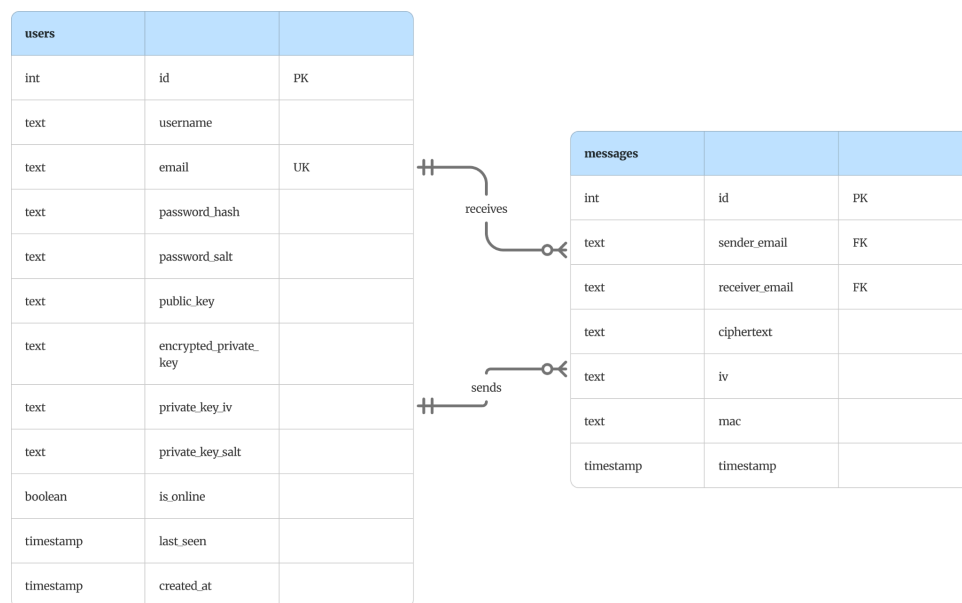
Gambar 3.1 Arsitektur client-server

Sistem terdiri dari dua komponen utama, yaitu Client dan Server, yang berkomunikasi melalui HTTPS/TLS dengan JWT sebagai mekanisme autentikasi.

Sisi klien menangani seluruh operasi kriptografi secara mandiri menggunakan Web Crypto API, meliputi enkripsi pesan dengan AES-256-GCM, pembentukan *shared secret* menggunakan ECDH, derivasi kunci melalui HKDF, dan verifikasi integritas pesan menggunakan HMAC-SHA256. Selain modul kriptografi, sisi klien juga terdiri dari komponen UI dan API layer yang menangani komunikasi dengan server.

Sisi server bertugas sebagai perantara yang menerima, menyimpan, dan meneruskan data terenkripsi tanpa mengetahui isi plaintext pesan. Server terdiri dari *middleware* keamanan (CORS, Helmet, JWT verification), *routes* dan *controllers* untuk tiga kelompok endpoint (`/api/auth`, `/api/users`, `/api/messages`), layer services yang mengandung logika bisnis, serta implementasi mandiri JWT library menggunakan algoritma ECDSA. Seluruh data disimpan pada basis data PostgreSQL.

3.2 Skema basis data



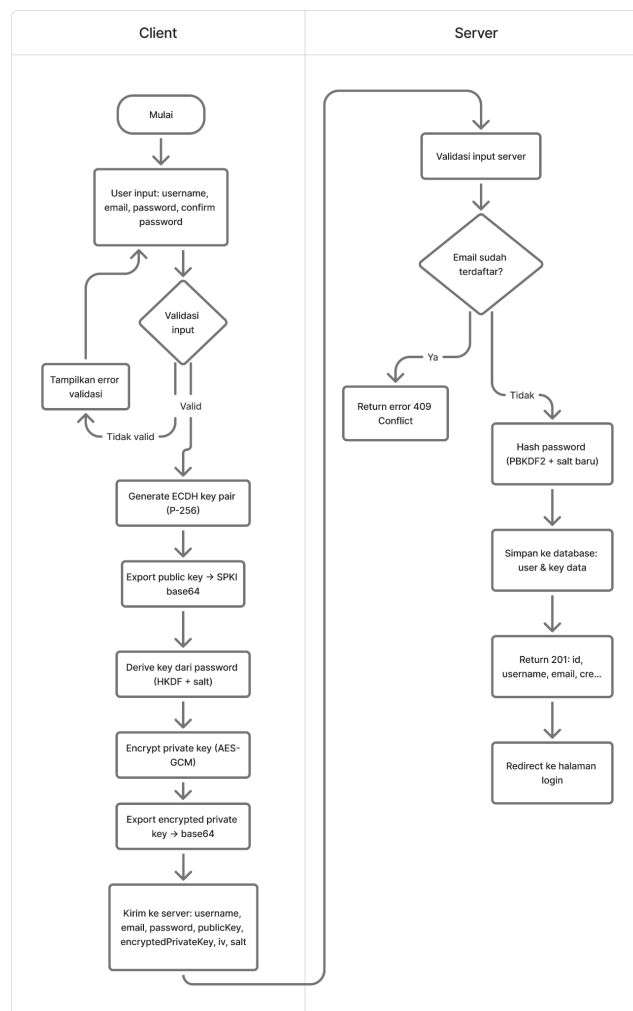
Gambar 3.2 ERD database

Tabel users menyimpan data pengguna meliputi informasi akun (username, email) dan kredensial keamanan (password_hash, password_salt), serta seluruh kunci kriptografis yang dibutuhkan untuk proses ECDH, yaitu public_key, encrypted_private_key, private_key_iv, dan private_key_salt. Tabel ini juga menyimpan status kehadiran pengguna melalui kolom is_online dan last_seen.

Tabel *messages* menyimpan seluruh pesan terenkripsi antar pengguna. Kolom *sender_email* dan *receiver_email* merupakan foreign key yang mereferensikan kolom *email* pada tabel *users*, membentuk relasi *sends* dan *receives*. Pesan disimpan dalam bentuk terenkripsi pada kolom *ciphertext* beserta *iv* yang digunakan pada proses enkripsi AES, serta *mac* untuk keperluan verifikasi integritas pesan.

Server tidak menyimpan plaintext pesan maupun kunci komunikasi, sehingga seluruh data sensitif yang tersimpan di basis data hanya dapat dibaca oleh klien yang memiliki kredensial yang sesuai.

3.3 Alur registrasi pengguna



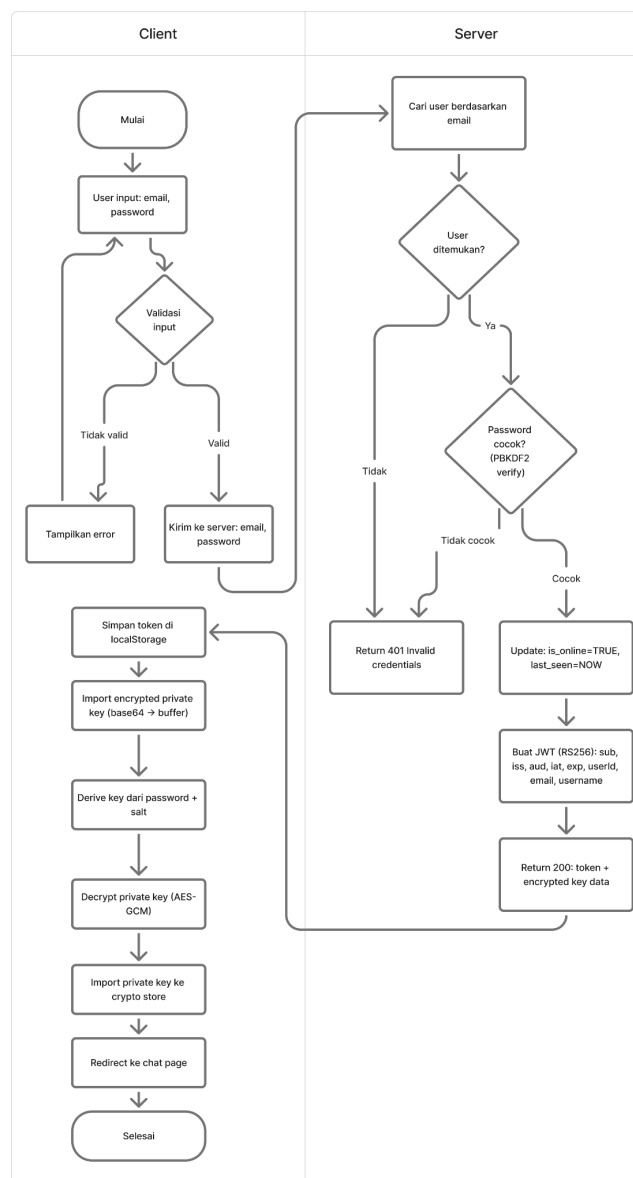
Gambar 3.3 Alur registrasi pengguna

Di sisi klien, pengguna memasukkan username, email, password, dan confirm password. Setelah validasi input berhasil, klien membangkitkan pasangan kunci ECDH dengan kurva P-256 dan mengeksport public key dalam format SPKI base64.

Selanjutnya, klien menurunkan kunci enkripsi dari *password* menggunakan HKDF dengan salt baru, kemudian menggunakan kunci tersebut untuk mengenkripsi private key menggunakan AES-GCM. Seluruh data yang dikirim ke server mencakup username, email, password, publicKey, encryptedPrivateKey, iv, dan salt.

Di sisi server, input divalidasi kembali dan dilakukan pengecekan apakah email sudah terdaftar. Apabila sudah terdaftar, server mengembalikan *error 409 Conflict*. Apabila belum, server melakukan hashing password menggunakan bcrypt dengan salt baru, menyimpan seluruh data pengguna beserta kunci kriptografis ke basis data, dan mengembalikan *response 201 Created* diikuti *redirect* ke halaman *login*.

3.4 Alur login dan autentikasi JWT

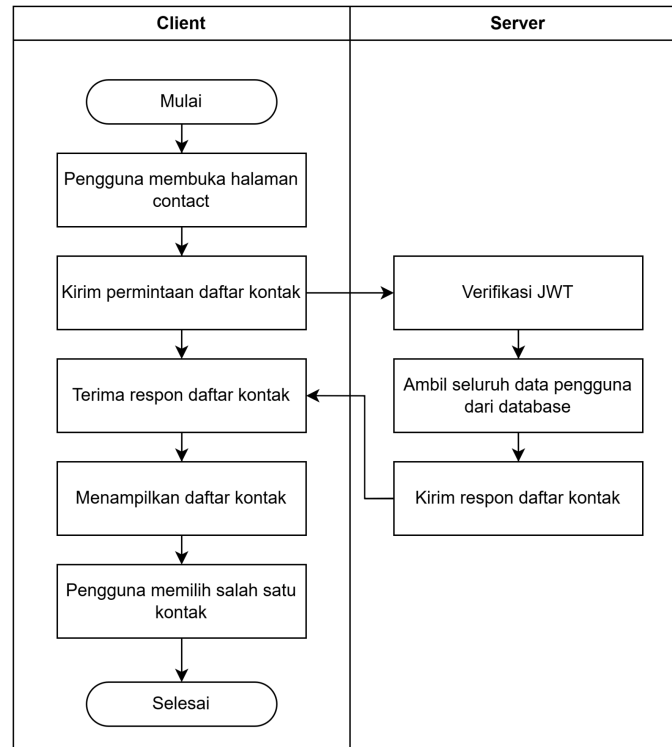


Gambar 3.4 Alur login dan autentikasi JWT

Di sisi klien, pengguna memasukkan email dan password. Setelah validasi input berhasil, data dikirim ke server. Server mencari pengguna berdasarkan email dan memverifikasi password menggunakan bcrypt. Apabila email tidak ditemukan atau password tidak cocok, server mengembalikan error 401 Invalid Credentials. Apabila berhasil, server memperbarui status pengguna menjadi `is_online = TRUE`, membuat JWT yang ditandatangani dengan algoritma ES256 memuat klaim `sub`, `iss`, `aud`, `iat`, `exp`, `userId`, dan `email`, kemudian mengembalikan *response* 200 beserta token dan data kunci terenkripsi.

Di sisi klien, token disimpan di `localStorage`, kemudian klien mengimpor `encryptedPrivateKey` dari base64 ke buffer, menurunkan kunci dekripsi dari password dan salt menggunakan HKDF, dan mendekripsi private key menggunakan AES-GCM. *Private key* yang berhasil dipulihkan kemudian diimpor ke crypto store untuk digunakan dalam proses *key exchange* ECDH, sebelum akhirnya pengguna diarahkan ke halaman chat.

3.5 Alur daftar kontak



Gambar 3.5 Diagram Alur Daftar Kontak

Setelah pengguna berhasil *login*, sistem mengarahkan pengguna ke halaman Contacts. Halaman ini menampilkan seluruh pengguna yang telah terdaftar pada aplikasi, kecuali akun pengguna itu sendiri. Alur pengambilan daftar kontak berjalan sebagai berikut:

1. Klien mengirim HTTP GET ke endpoint

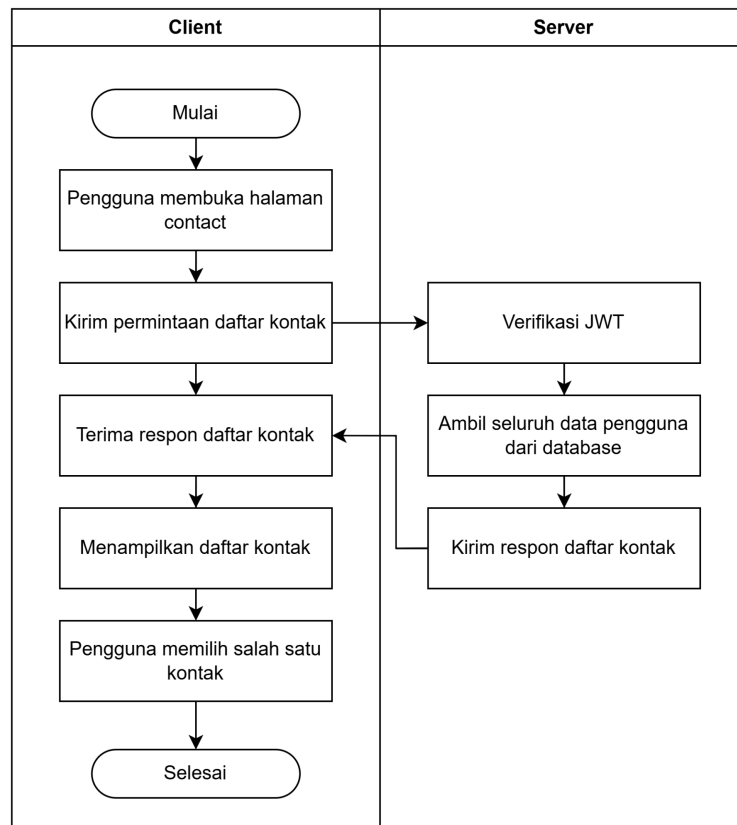
GET /api/users/contacts Authorization: Bearer <JWT>
--

2. Server memverifikasi JWT yang disertakan pada *header* Authorization menggunakan *middleware* autentikasi.
3. Jika JWT valid, server mengambil seluruh data pengguna dari basis data, kecuali akun sendiri.
4. Server mengembalikan array objek yang berisi email setiap pengguna.
5. Klien merender daftar tersebut pada komponen ContactList/AllContactsList.
6. Pengguna memilih salah satu kontak untuk memulai percakapan, yang kemudian memicu proses *key exchange* ECDH.

File relevan:

- client/src/api/userApi.js (fungsi getContacts())
- client/src/pages/ContactsPage.jsx.

3.6 Alur pembentukan kunci komunikasi



Gambar 3.6 Diagram Alur Key Exchange

Ketika pengguna memilih kontak untuk memulai percakapan, sistem menjalankan proses pembentukan kunci komunikasi menggunakan ECDH dan HKDF. Alur prosesnya berjalan sebagai berikut:

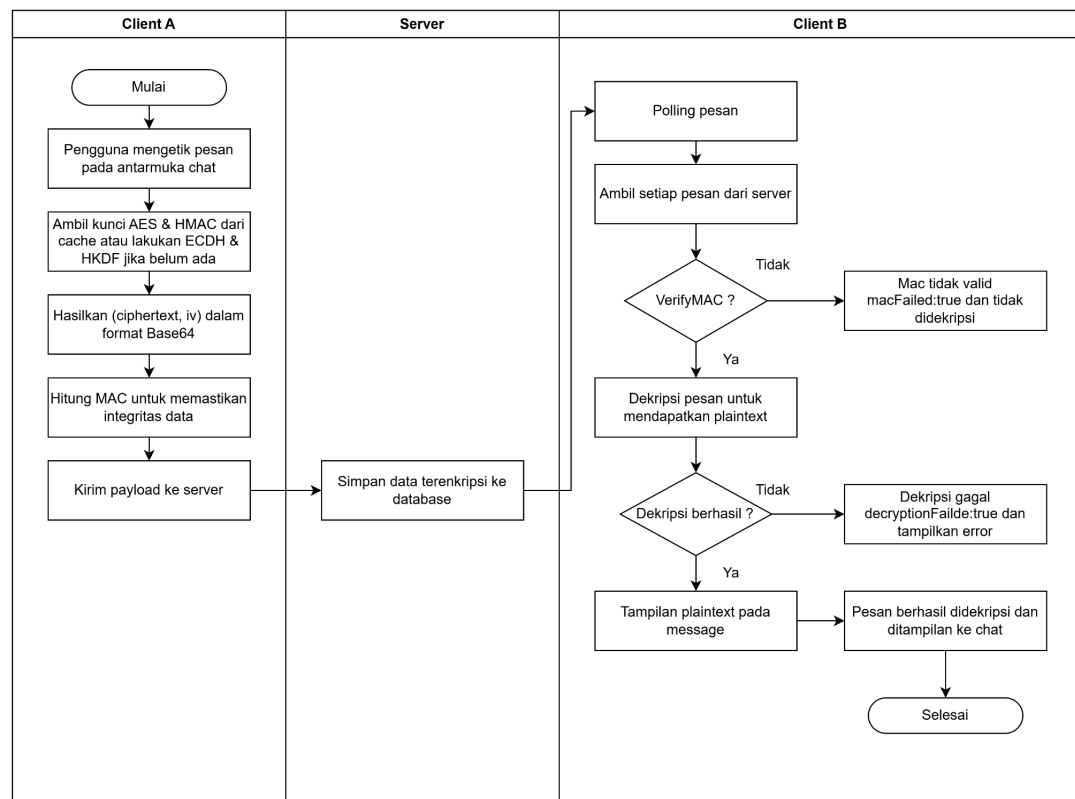
1. Pengguna A memilih Pengguna B dari daftar kontak untuk memulai percakapan.
2. Klien A mengirim HTTP GET ke *endpoint* `GET/api/users/:email/public-key` untuk mengambil *public key* milik Pengguna B dari server.
3. Klien A juga mengambil *encrypted private key* milik sendiri dari server melalui *endpoint* `GET /api/users/me/private-key`.
4. Klien A mendekripsi *encrypted private key* menggunakan kunci yang diturunkan dari *password* pengguna melalui `PBKDF2 → AES-256-GCM`, sehingga diperoleh *private key ECDH* dalam bentuk *CryptoKey*.

5. Klien A menghitung *shared secret* menggunakan operasi ECDH: `deriveBits(privateKey_A, publicKey_B)` melalui Web Crypto API.
6. *Shared secret* diproses menggunakan HKDF-SHA-256 untuk menghasilkan dua kunci terpisah:
 - Kunci AES-256-GCM dengan nilai `info = "chat-aes-key-v1"` untuk enkripsi pesan.
 - Kunci HMAC-SHA-256 dengan nilai `info = "chat-hmac-key-v1"` untuk verifikasi integritas pesan.
7. Kedua kunci di-*cache* di memori browser melalui `authStorage` dan akan hilang saat sesi berakhir.
8. Proses yang sama terjadi secara simetris di sisi Pengguna B, `ECDH(privateKey_B, publicKey_A)` menghasilkan kunci AES dan HMAC yang identik.
9. Server hanya berperan menyimpan dan meneruskan *public key*, dan tidak pernah mengetahui *shared secret* maupun kunci komunikasi.

File relevan:

- `client/src/crypto/ecdh.js` (fungsi `computeSharedSecret()`)
- `client/src/crypto/hkdf.js` (fungsi `deriveAESKey()`, `deriveHMACKey()`)
- `client/src/crypto/passwordKey.js` (fungsi `decryptPrivateKey()`)
- `client/src/storage/authStorage.js` (fungsi `cacheAesKey()`, `cacheHmacKey()`)

3.7 Alur pengiriman pesan terenkripsi



Gambar 3.7 Diagram Alur Pengiriman Pesan Terenkripsi

Pengiriman pesan pada SecureChat sepenuhnya dilakukan secara *end-to-end encrypted*. Server tidak pernah menyentuh kunci komunikasi maupun *plaintext* pesan. Berikut alur lengkapnya:

Sisi Pengirim:

1. Pengguna mengetik pesan pada antarmuka chat.
2. Klien memanggil `getOrCreateKeys(contact)` untuk mendapatkan kunci AES dan HMAC yang sudah di-*cache*, atau melakukan ECDH + HKDF jika belum ada.
3. Klien memanggil `aesEncrypt(aesKey, plaintext)` yang menghasilkan `{ ciphertext, iv }` dalam format Base64.
4. Klien menghitung MAC dengan `computeMAC(hmacKey, ciphertext)` untuk memastikan integritas pesan.
5. Klien mengirim *payload* JSON ke server melalui `sendMessage()`:

```
POST /api/messages
Authorization: Bearer <JWT>

{
  "receiverEmail": "bob@example.com",
  "ciphertext": "<base64>",
  "iv":          "<base64>",
  "mac":         "<base64>"
}
```

6. Server menyimpan data terenkripsi ke basis data tanpa mengetahui isi pesan.

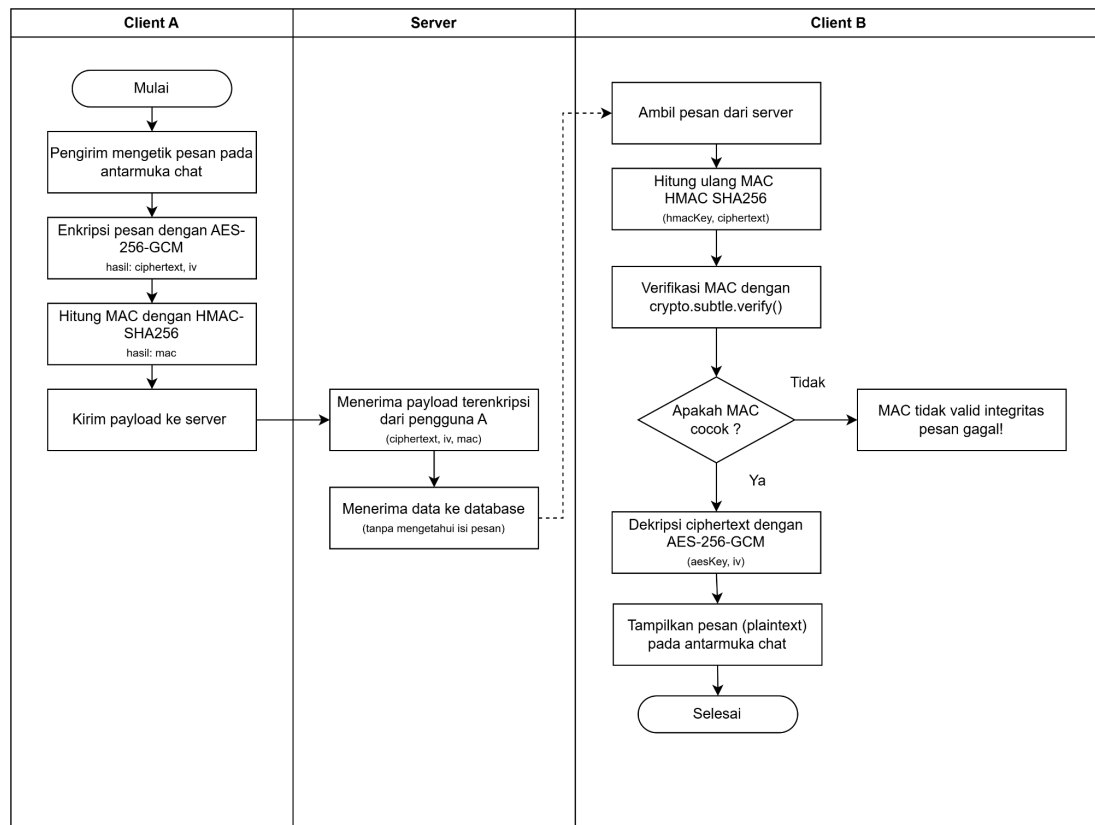
Sisi Penerima:

1. Klien melakukan polling setiap 3 detik ke *endpoint* GET `/api/messages/:contactEmail`.
2. Untuk setiap pesan, klien pertama-tama memverifikasi MAC menggunakan `verifyMAC(hmacKey, ciphertext, mac)`. Jika gagal, pesan ditandai `macFailed: true` dan tidak didekripsi.
3. Jika MAC valid, klien memanggil `aesDecrypt(aesKey, ciphertext, iv)` untuk mendapatkan plaintext.
4. Jika dekripsi gagal (misalnya karena data korup), pesan ditandai `decryptionFailed: true` dan antarmuka menampilkan pesan *error*.
5. Plaintext yang berhasil didekripsi ditampilkan pada komponen `MessageBubble`.

File relevan:

- `client/src/pages/ContactsPage.jsx` (fungsi `handleSend()` dan `fetchMessages()`)
- `client/src/api/messageApi.js`.

3.8 Alur verifikasi MAC



Gambar 3.8 Diagram Alur Verifikasi MAC

Setiap pesan yang dikirim disertai dengan MAC untuk memastikan integritas dan autentisitas pesan. Alur verifikasi MAC berjalan sebagai berikut:

Sisi Pengirim:

1. Pengirim mengetik pesan pada antarmuka chat.
2. Klien mengenkripsi plaintext menggunakan AES-256-GCM dengan kunci AES dari hasil *key exchange*, menghasilkan *ciphertext* dan *IV*.
3. Klien menghitung MAC dengan HMAC-SHA256(hmacKey, ciphertext) menggunakan kunci HMAC dari hasil *key exchange*.
4. Klien mengirim payload { ciphertext, iv, mac } ke server melalui endpoint POST /api/messages.

Sisi Penerima:

1. Klien penerima mengambil pesan dari server melalui endpoint GET /api/messages/:contactEmail.

Gambar 3.9 Diagram Arsitektur Container Docker

Aplikasi dikemas menggunakan Docker untuk memastikan konsistensi lingkungan eksekusi di berbagai perangkat. Seluruh konfigurasi diatur melalui `docker-compose.yml` dengan tiga service utama:

1. Server memverifikasi JWT yang disertakan pada header Authorization menggunakan *middleware* autentikasi sebelum memproses setiap permintaan.
2. Service db (PostgreSQL) dijalankan pertama kali karena service lain bergantung padanya. Data disimpan pada Docker volume agar tetap persisten meski *container* di-restart.
3. Service server (Node.js/Express) dibangun dari `server/Dockerfile` dan terhubung ke database menggunakan nama service db sebagai hostname. Service ini dikonfigurasi dengan `depends_on: db` agar menunggu database siap sebelum dijalankan.
4. Service client (React/Vite) dibangun dari `client/Dockerfile`. Client berkomunikasi dengan server melalui URL yang dikonfigurasi via environment variable `VITE_API_URL`.
5. Ketiga service berada dalam satu Docker network internal sehingga dapat saling berkomunikasi menggunakan nama *service* sebagai *hostname*.
6. Hanya port yang diperlukan yang di-expose ke host, port 5173 untuk *client* dan port 3000 untuk server.
7. Seluruh aplikasi dapat dijalankan dengan satu perintah: `docker compose up --build`.

File relevan:

- `docker-compose.yml`
- `client/Dockerfile`
- `server/Dockerfile`

IV. Implementasi

4.1 Implementasi frontend

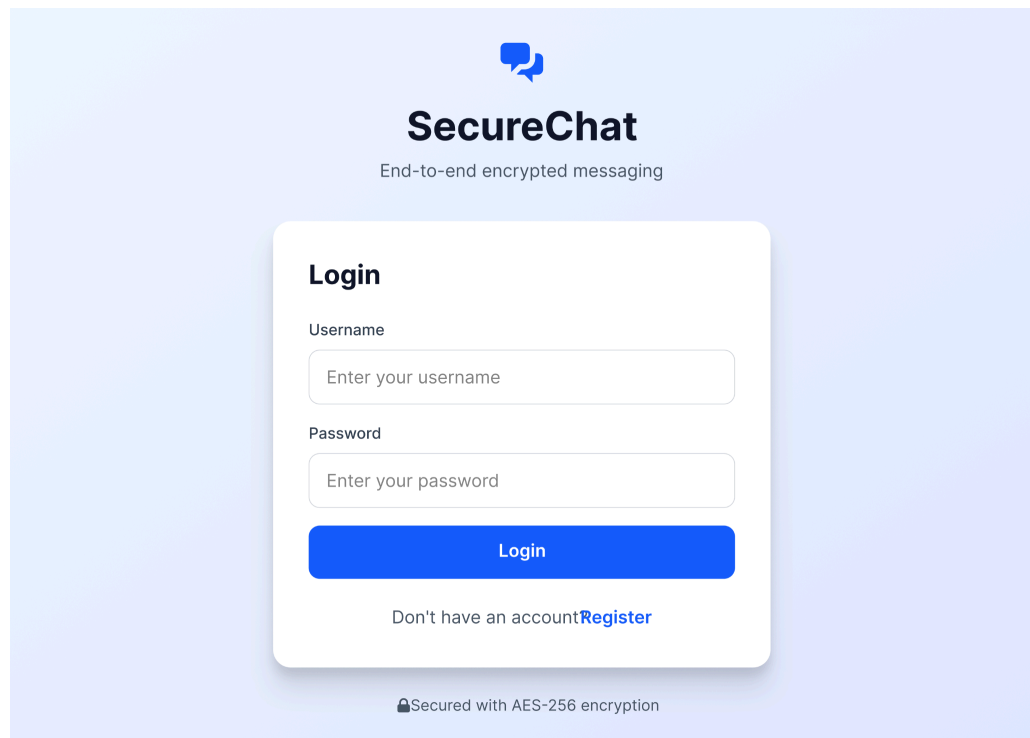
Frontend SecureChat dibangun menggunakan React dengan Vite sebagai *bundler*. *Styling* menggunakan Tailwind CSS. Struktur halaman terdiri dari empat halaman utama:

Tabel 4.1 Halaman Utama Chat

Halaman	File	Fungsi
Registrasi	RegisterPage.jsx	Form registrasi, pembangkitan kunci ECDH, enkripsi private key
Login	LoginPage.jsx	Autentikasi, dekripsi private key, penyimpanan CryptoKey di memori
Chat	ChatPage.jsx	Redirect ke contacts untuk kompatibilitas routing
Daftar Kontak	ContactsPage.jsx	Daftar kontak, key exchange, polling pesan, enkripsi/dekripsi

Komponen pendukung tersimpan di `client/src/components/`, meliputi `ChatBox.jsx`, `ContactList.jsx`, `ErrorMessage.jsx`, `MessageBubble.jsx`, dan `Navbar.jsx`. *Routing* menggunakan React Router dengan proteksi akses melalui `ProtectedRoute.jsx` (`client/src/routes/`) yang memvalidasi keberadaan JWT sebelum mengizinkan akses ke halaman terproteksi.

State manajemen dilakukan secara lokal menggunakan React hooks (`useState`, `useEffect`). Kunci kriptografi (`CryptoKey`) disimpan di memori sesi melalui `authStorage.js` (`client/src/storage/`) dan tidak pernah dipersistensikan ke `localStorage` untuk alasan keamanan.



The image shows the login interface for SecureChat. At the top, there is a blue speech bubble icon, the text "SecureChat", and the tagline "End-to-end encrypted messaging". The main form is titled "Login" and contains two input fields: "Username" with the placeholder "Enter your username" and "Password" with the placeholder "Enter your password". Below these fields is a blue "Login" button. At the bottom of the form, there is a link "Don't have an account? Register". A security notice at the very bottom states "Secured with AES-256 encryption".


SecureChat
End-to-end encrypted messaging

Login

Username

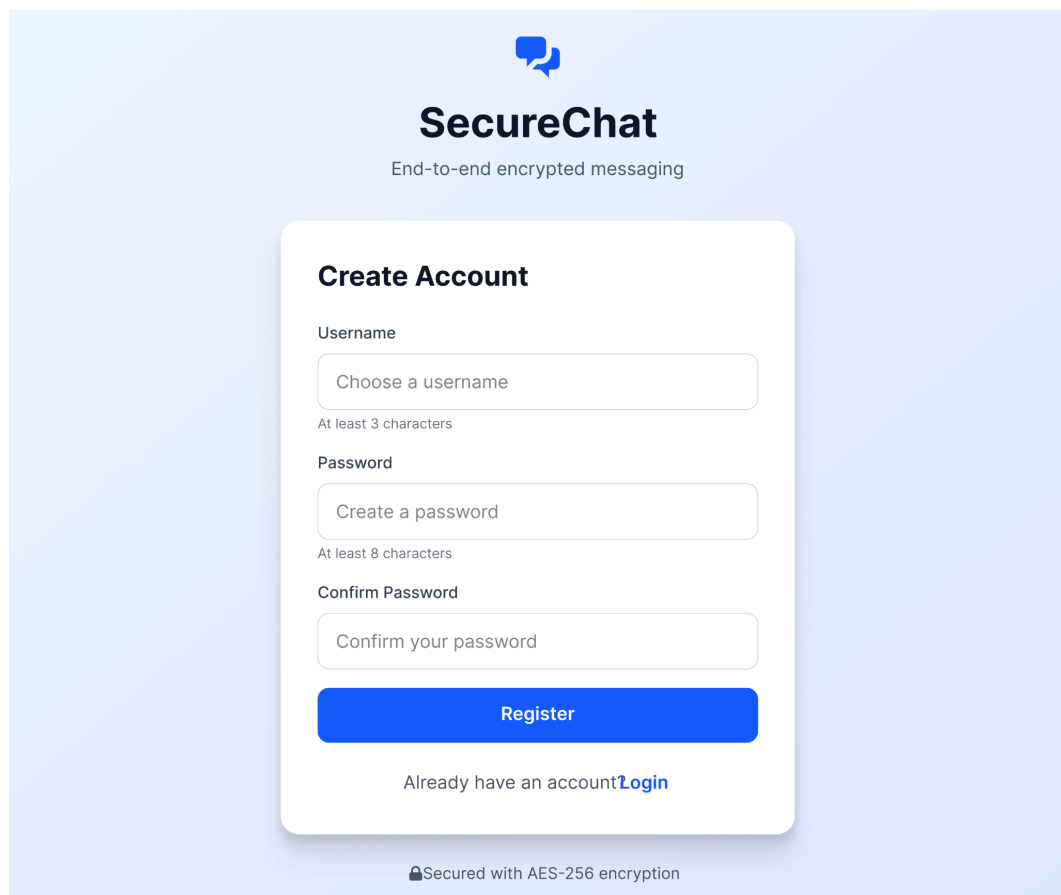
Password

Login

Don't have an account? [Register](#)

 Secured with AES-256 encryption

Gambar 4.1 Antarmuka halaman Login



The image shows the registration interface for SecureChat. At the top, there is a blue speech bubble icon, the text "SecureChat", and the tagline "End-to-end encrypted messaging". The main form is titled "Create Account" and contains three input fields: "Username" with the placeholder "Choose a username" and a note "At least 3 characters", "Password" with the placeholder "Create a password" and a note "At least 8 characters", and "Confirm Password" with the placeholder "Confirm your password". Below these fields is a blue "Register" button. At the bottom of the form, there is a link "Already have an account? Login". A security notice at the very bottom states "Secured with AES-256 encryption".


SecureChat
End-to-end encrypted messaging

Create Account

Username

At least 3 characters

Password

At least 8 characters

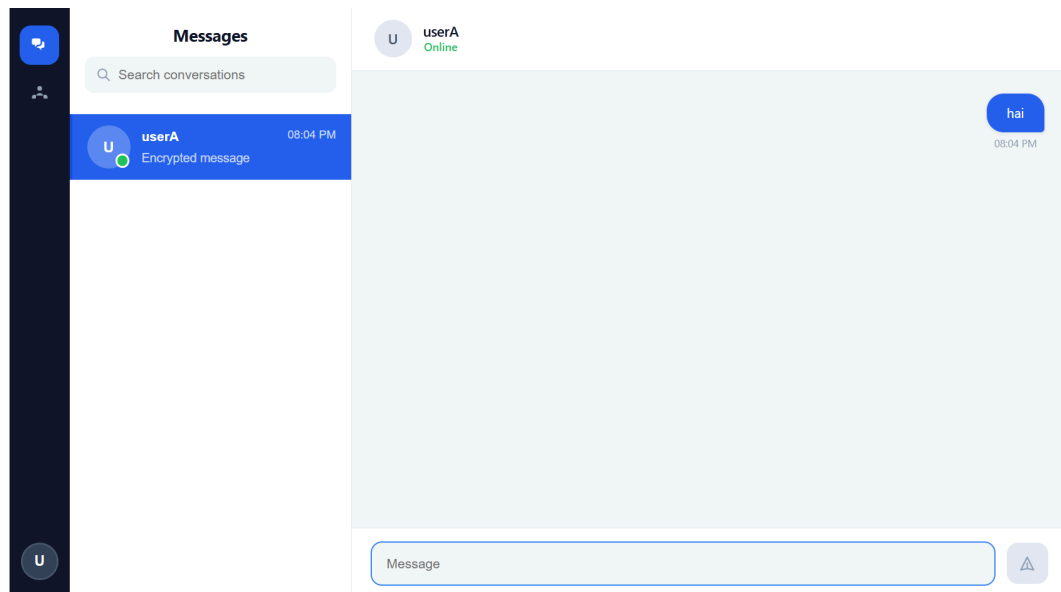
Confirm Password

Register

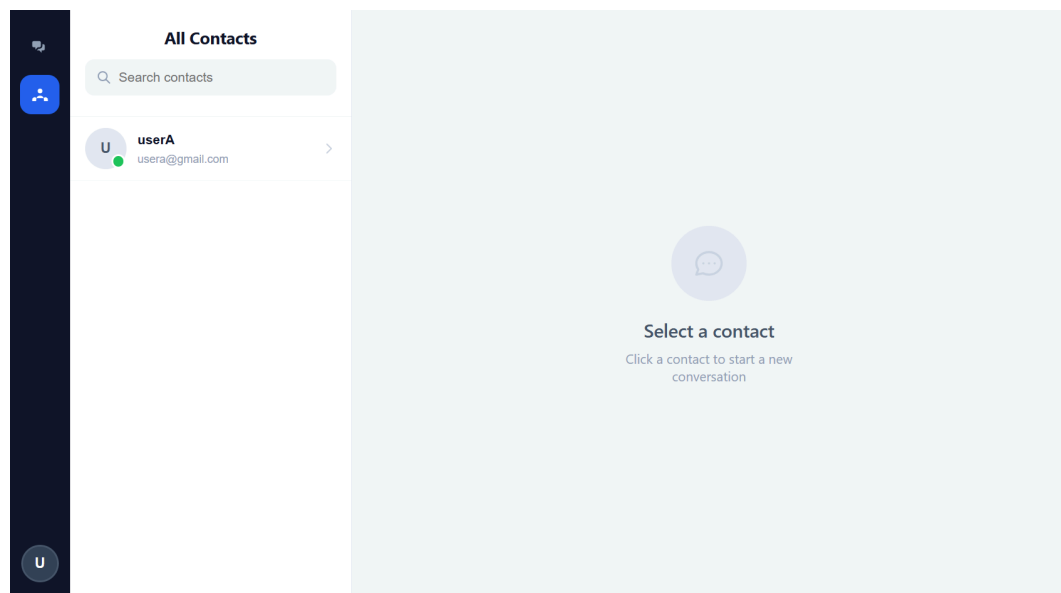
Already have an account? [Login](#)

 Secured with AES-256 encryption

Gambar 4.2 Antarmuka halaman Register



Gambar 4.3 Antarmuka halaman Chat



Gambar 4.4 Antarmuka halaman Daftar Kontak

4.2 Implementasi API Client

Semua komunikasi antara klien dan server dikelola melalui modul-modul API di direktori `client/src/api/`. Seluruh request yang membutuhkan autentikasi menyertakan JWT pada *header*. Token diambil dari storage melalui fungsi `getToken()` dari `authStorage.js`.

4.2.1 userApi.js

File `userApi.js` menyediakan tiga fungsi utama:

Tabel 4.2 Fungsi pada `userApi.js`

Fungsi	Endpoint	Keterangan
<code>getContacts()</code>	GET <code>/api/users/contacts</code>	Mengambil daftar semua pengguna terdaftar (kecuali diri sendiri)
<code>getPublicKey(email)</code>	GET <code>/api/users/:email/public-key</code>	Mengambil <i>public key</i> ECDH milik pengguna lain untuk <i>key exchange</i>
<code>getMyPrivateKeyData()</code>	GET <code>/api/users/me/private-key</code>	Mengambil <i>encrypted private key</i> , IV, dan salt untuk proses <i>login</i>

Contoh implementasi `getPublicKey()` yang digunakan dalam proses *key exchange*:

```
# Implementasi getPublicKey() pada userApi.js

// userApi.js
export async function getPublicKey(email) {
  const res = await fetch(
    `${BASE_URL}/api/users/${encodeURIComponent(email)}/public-key`,
    { headers: authHeaders() }
  );
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || 'Failed to fetch public key');
  return data; // { publicKey: '<base64>' }
}
```

4.2.2 messageApi.js

File `messageApi.js` menyediakan dua fungsi untuk operasi pesan:

Tabel 4.3 Fungsi pada `messageApi.js`

Fungsi	Endpoint	Keterangan
<code>sendMessage(payload)</code>	POST <code>/api/messages</code>	Mengirim pesan terenkripsi (ciphertext, iv, mac, receiverEmail)
<code>getMessages(contactEmail)</code>	GET <code>/api/messages/:email</code>	Mengambil seluruh pesan dengan kontak tertentu

Contoh struktur *payload* yang dikirimkan oleh `sendMessage()`:

```
# Implementasi sendMessage() pada messageApi.js

// messageApi.js
export async function sendMessage(payload) {
  const res = await fetch(`${BASE_URL}/api/messages`, {
    method: 'POST',
    headers: authHeaders(),
    body: JSON.stringify(payload),
    // payload: { receiverEmail, ciphertext, iv, mac }
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || 'Failed to send message');
  return data;
}
```

4.2.3 authApi.js

File `authApi.js` menyediakan tiga fungsi untuk operasi autentikasi:

Tabel 4.4 Fungsi pada `authApi.js`

Fungsi	Endpoint	Keterangan
register(payload)	POST /api/auth/register	Mengirim data registrasi (email, hashed password, salt, public key, encrypted private key)
login(payload)	POST /api/auth/login	Mengirim kredensial dan menerima JWT sebagai respons
logout()	POST /api/auth/logout	Menghapus JWT dari cookie sesi

Contoh implementasi `login()` yang digunakan dalam proses autentikasi:

```
# Implementasi login() pada authApi.js

// authApi.js
export async function login(payload) {
  const res = await fetch(`${BASE_URL}/api/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include', // agar cookie JWT tersimpan otomatis
    body: JSON.stringify(payload),
    // payload: { email, password }
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || 'Login failed');
  return data;
}
```


4.3 Implementasi backend API

A. Authentication Endpoints

POST /api/auth/register

Endpoint ini digunakan untuk mendaftarkan pengguna baru ke dalam sistem. Klien mengirimkan data berupa *username*, *email*, *password*, *publicKey*, *encryptedPrivateKey*, beserta parameter kriptografis seperti *iv* dan *salt*. Proses yang dijalankan adalah sebagai berikut:

1. Validasi seluruh input termasuk format *email*, panjang *password*, dan kelengkapan kunci kriptografis.
2. Pengecekan bahwa *email* belum terdaftar di basis data.
3. Hashing *password* menggunakan *bcrypt*.
4. Penyimpanan data pengguna beserta kunci kriptografis ke basis data.

Endpoint ini mengembalikan status 201 Created beserta data pengguna yang berhasil dibuat (*id*, *username*, *email*, *created_at*). Apabila terjadi kesalahan, sistem mengembalikan status 400 untuk kegagalan validasi input atau status 409 Conflict apabila *email* sudah terdaftar sebelumnya.

POST /api/auth/login

Endpoint ini digunakan untuk melakukan autentikasi pengguna. Klien mengirimkan *email* dan *password*. Proses yang dijalankan adalah sebagai berikut:

1. Validasi input.
2. Pencarian data pengguna berdasarkan *email* di basis data.
3. Verifikasi *password* menggunakan *bcrypt.compare()*.
4. Penerbitan JWT token yang ditandatangani menggunakan algoritma ES256.
5. Pembaruan status kehadiran pengguna (*is_online* = TRUE, *last_seen* = CURRENT_TIMESTAMP).

Endpoint ini mengembalikan status 200 OK beserta token JWT dan data pengguna, termasuk kunci kriptografis (publicKey, encryptedPrivateKey, kdfSalt, kdfParams) yang dibutuhkan klien untuk memulihkan private key ECDH. Apabila terjadi kesalahan, sistem mengembalikan status 400 untuk kegagalan validasi atau status 401 Unauthorized apabila email tidak ditemukan atau *password* tidak sesuai.

POST /api/auth/logout (Authenticated)

Endpoint ini digunakan untuk mengakhiri sesi pengguna. Request wajib menyertakan header Authorization: Bearer <JWT_TOKEN>. Proses yang dijalankan adalah pembaruan status kehadiran pengguna menjadi is_online = FALSE.

B. User Endpoints

GET /api/users/contacts (Authenticated)

Endpoint ini digunakan untuk mengambil daftar seluruh pengguna terdaftar lainnya yang dapat dikontaki. Response berupa list pengguna dengan informasi username, email, publicKey, isOnline, dan lastSeen. Data publicKey pada endpoint ini digunakan oleh klien untuk memulai proses key exchange ECDH ketika hendak berkomunikasi dengan pengguna lain.

C. Message Endpoints

POST /api/messages/send (Authenticated)

Endpoint ini digunakan untuk mengirimkan pesan terenkripsi kepada pengguna lain. Klien mengirimkan receiverEmail, ciphertext, iv, dan mac. Server hanya menyimpan data terenkripsi ke basis data tanpa mengetahui isi *plaintext* pesan.

GET /api/messages/:receiverEmail (Authenticated)

Endpoint ini digunakan untuk mengambil riwayat pesan antara pengguna yang sedang login dengan pengguna yang ditentukan oleh parameter receiverEmail. Response berupa list pesan terenkripsi yang selanjutnya akan didekripsi di sisi klien.

4.4 Implementasi custom JWT library

Library JWT kustom diimplementasikan pada direktori `server/src/jwt-lib/`. Library ini mengimplementasikan fungsi `sign` dan `verify` sesuai standar RFC 7519 dan RFC 7515 (JWS).

4.4.1 Fungsi *Sign* (`server/src/jwt-lib/sign.js`)

Fungsi `sign` menerima parameter `header`, `claims`, `payload`, dan `privateKey`. Proses dimulai dengan validasi header dan algoritma, kemudian membangun payload akhir dengan menggabungkan payload dan claims di mana nilai claims diprioritaskan jika terdapat *key* yang sama. Nilai `iat` ditambahkan secara otomatis jika belum ada.

Header dan payload masing-masing di-encode ke Base64URL, lalu signing input `base64url(header).base64url(payload)` ditandatangani menggunakan ECDSA melalui `crypto.sign()` dengan opsi `dsaEncoding: "ieee-p1363"` sesuai standar JWT (bukan format DER default Node.js). Token akhir dikembalikan dalam format `base64url(header).base64url(payload).base64url(signature)`.

```
// server/src/jwt-lib/sign.js (cuplikan)

const signingInput = `${headerB64}.${payloadB64}`;

signatureBuffer = crypto.sign(hashOpts.hash, Buffer.from(signingInput), {
  key: keyObject,
  dsaEncoding: "ieee-p1363",
});

return `${signingInput}.${encode(signatureBuffer)}`;
```

4.4.2 Fungsi *Verify* (`server/src/jwt-lib/verify.js`)

Fungsi `verify` memvalidasi token sesuai RFC 7519 Section 7.2. Token dipisahkan menjadi tiga segmen, kemudian *header* dan *payload* di-*decode* dari Base64URL dan di-parse sebagai JSON. Signature diverifikasi menggunakan `crypto.verify()` dengan `dsaEncoding: "ieee-p1363"`. Setelah signature valid, klaim waktu (`exp`, `nbf`) dan klaim identitas (`iss`, `sub`, `aud`, `jti`) divalidasi sesuai parameter `options`. Fungsi mengembalikan `{ header, payload, signature }` jika seluruh validasi berhasil, atau melempar `JWTErrror` jika gagal.

```
// server/src/jwt-lib/verify.js (cuplikan)
isValid = crypto.verify(
  hashOpts.hash,
  Buffer.from(signingInput),
  { key: keyObject, dsaEncoding: "ieee-p1363" },
  signatureBuffer
);

if (!isValid) throw new JWTError(ERROR_MESSAGES.INVALID_SIGNATURE);

// Validasi klaim waktu
const now = Math.floor(Date.now() / 1000);
if (!options.ignoreExp && payload.exp !== undefined) {
  if (now >= payload.exp) throw new
JWTError(ERROR_MESSAGES.TOKEN_EXPIRED);
}
if (!options.ignoreNbf && payload.nbf !== undefined) {
  if (now < payload.nbf) throw new
JWTError(ERROR_MESSAGES.TOKEN_NOT_YET_VALID);
}
```

4.4.3 Modul Pendukung

- **base64url.js**
mengimplementasikan fungsi `encode()` dan `decode()` untuk konversi Base64URL sesuai RFC 4648 §5, termasuk penggantian karakter `+` → `-`, `/` → `_`, dan penanganan padding `=`.
- **errors.js**
mendefinisikan class `JWTError` dan konstanta `ERROR_MESSAGES` berisi pesan error standar untuk seluruh kondisi error pada proses *sign* dan *verify*, sehingga pesan error konsisten di seluruh library.
- **keyLoader.js**
menyediakan fungsi `loadPrivateKey()` dan `loadPublicKey()` untuk memuat PEM key ke `KeyObject` Node.js, serta fungsi `algToHashOptions()` yang memetakan nama algoritma JWT (ES256/ES384/ES512) ke nama hash dan nama kurva OpenSSL yang digunakan Node.js.

File relevan:

- `server/src/jwt-lib/base64url.js`
- `server/src/jwt-lib/errors.js`
- `server/src/jwt-lib/keyLoader.js`

4.5 Implementasi database

Database yang digunakan adalah PostgreSQL dengan mekanisme *connection pooling* melalui *library* node-postgres untuk mengelola koneksi secara efisien. *Connection pooling* memungkinkan sistem untuk melakukan reuse koneksi yang sudah ada sehingga mengurangi overhead pembukaan koneksi baru pada setiap request, mendukung konkurensi tinggi, serta menyediakan mekanisme automatic reconnection apabila koneksi terputus.

4.5.1 Skema database

Tabel 4.5 Tabel users

Kolom	Tipe	Constraint	Deskripsi
id	SERIAL	PRIMARY KEY	<i>Unique identifier</i>
username	TEXT	NOT NULL	Nama tampilan pengguna
email	TEXT	NOT NULL, UNIQUE	Email sebagai <i>login identifier</i>
password_hash	TEXT	NOT NULL	Hash password menggunakan bcrypt
password_salt	TEXT	NOT NULL	Salt untuk PBKDF2 <i>key derivation</i>
public_key	TEXT	NOT NULL	ECDH <i>public key</i> (base64)
encrypted_private_key	TEXT	NOT NULL	ECDH <i>private key</i> terenkripsi (base64)
private_key_iv	TEXT	NOT NULL	IV untuk enkripsi <i>private key</i> (base64)
private_key_salt	TEXT	NULLABLE	Salt untuk <i>private key derivation</i>
is_online	BOOLEAN	DEFAULT FALSE	Status kehadiran pengguna
last_seen	TIMESTAMP	NULLABLE	Timestamp terakhir pengguna aktif
created_at	TIMESTAMP	DEFAULT NOW()	Waktu registrasi

Tabel 4.6 Tabel messages

Kolom	Tipe	Constraint	Deskripsi
id	SERIAL	PRIMARY KEY	<i>Unique identifier</i>
sender_email	TEXT	NOT NULL, FK	Email pengirim
receiver_email	TEXT	NOT NULL, FK	Email penerima
ciphertext	TEXT	NOT NULL	Pesan terenkripsi AES-256-GCM (base64)
iv	TEXT	NOT NULL	IV untuk AES-256-GCM (base64)
mac	TEXT	NULLABLE	HMAC-SHA256 untuk verifikasi integritas
timestamp	TIMESTAMP	DEFAULT NOW()	Waktu pesan dikirim

4.6 Implementasi enkripsi pesan

Implementasi enkripsi dan dekripsi pesan terdapat pada file `client/src/crypto/aes.js` menggunakan Web Crypto API [5]. Varian yang digunakan adalah **AES-256-GCM**.

4.6.1 Enkripsi (aesEncrypt)

Fungsi `aesEncrypt()` menerima `CryptoKey` AES-256 dan string *plaintext*, kemudian mengembalikan `ciphertext` dan `iv` keduanya dalam format Base64.

```
# Fungsi aesEncrypt() pada client/src/crypto/aes.js

// aes.js
export async function aesEncrypt(key, plaintext) {
  // 1. Bangkitkan IV acak 96-bit (12 byte) untuk setiap pesan
  const iv = crypto.getRandomValues(new Uint8Array(12));

  // 2. Encode plaintext string ke byte
  const encoded = new TextEncoder().encode(plaintext);

  // 3. Enkripsi dengan AES-256-GCM via Web Crypto API
  const ciphertextBuffer = await crypto.subtle.encrypt(
    { name: 'AES-GCM', iv },
    key,
    encoded
  );
}
```

```
// 4. Kembalikan hasil dalam format Base64
return {
  ciphertext: bufferToBase64(ciphertextBuffer),
  iv: bufferToBase64(iv.buffer),
};
}
```

IV dibangkitkan secara acak setiap kali enkripsi dilakukan menggunakan `crypto.getRandomValues()` agar setiap pesan memiliki IV yang unik dan mencegah serangan *nonce reuse* yang berbahaya pada mode GCM [4].

4.6.2 Dekripsi Pesan (`aesDecrypt`)

Fungsi `aesDecrypt()` menerima `CryptoKey`, ciphertext Base64, dan IV Base64, kemudian mengembalikan plaintext string. Fungsi melempar `DOMException` jika dekripsi gagal baik karena kunci salah maupun data korup yang ditangkap oleh pemanggil untuk menampilkan penanda *error*.

```
# Fungsi aesDecrypt() pada client/src/crypto/aes.js

// aes.js
export async function aesDecrypt(key, ciphertext, iv) {
  // 1. Konversi Base64 ke ArrayBuffer
  const ciphertextBuffer = base64ToBuffer(ciphertext);
  const ivBuffer = base64ToBuffer(iv);

  // 2. Dekripsi - GCM otomatis memverifikasi authentication tag
  // Jika tag tidak cocok atau data korup, Promise di-reject
  const plaintextBuffer = await crypto.subtle.decrypt(
    { name: 'AES-GCM', iv: new Uint8Array(ivBuffer) },
    key,
    ciphertextBuffer
  );

  // 3. Decode byte ke string UTF-8
  return new TextDecoder().decode(plaintextBuffer);
}
```

4.6.3 Import Kunci AES (`importAesKey`)

Fungsi `importAesKey()` mengimpor raw key 32-byte (dihasilkan oleh HKDF) menjadi `CryptoKey` yang siap digunakan untuk enkripsi dan dekripsi:

```
# Fungsi importAesKey() pada client/src/crypto/aes.js

// aes.js
export async function importAesKey(rawKey) {
  return crypto.subtle.importKey(
    'raw',
    rawKey,
    { name: 'AES-GCM' },
    false, // tidak dapat diekstrak kembali
    ['encrypt', 'decrypt']
  );
}
```

4.6.4 Integrasi Enkripsi pada Pengiriman Pesan

Di `ContactsPage.jsx`, fungsi `handleSend()` mengorkestrasikan seluruh alur enkripsi sebelum pesan dikirim ke server. Kunci AES dan HMAC diambil atau diturunkan melalui , kemudian pesan dienkripsi dan MAC dihitung sebelum *payload* dikirim:

```
# Fungsi handleSend() pada ContactsPage.jsx

// ContactsPage.jsx - handleSend()
async function handleSend(text) {
  try {
    // 1. Ambil kunci dari cache atau turunkan baru via ECDH + HKDF
    const { aesKey, hmacKey } = await getOrDeriveKeys(selectedContact);

    // 2. Enkripsi plaintext → { ciphertext, iv }
    const { ciphertext, iv } = await aesEncrypt(aesKey, text);

    // 3. Hitung MAC untuk integritas pesan
    const mac = await computeMAC(hmacKey, ciphertext);

    // 4. Kirim payload terenkripsi ke server
    await sendMessage({ receiverEmail: selectedContact.email,
      ciphertext, iv, mac });

    // 5. Optimistic update - tampilkan pesan langsung
    setMessages(prev => [...prev, {
      senderEmail: myEmail, ciphertext, iv, mac,
      timestamp: new Date().toISOString(),
      isMine: true, plaintext: text, decryptionFailed: false
    }]);
  } catch (e) {
    setPageError(e.message);
  }
}
```


4.6.5 Integrasi Dekripsi pada Penerimaan Pesan

Fungsi dipanggil setiap 3 detik oleh interval polling. Dekripsi seluruh pesan dilakukan secara paralel menggunakan . Setiap pesan diproses melalui dua lapisan verifikasi: MAC terlebih dahulu, baru dekripsi AES.

```
# Dekripsi paralel pada fetchMessages() di ContactsPage.jsx

// ContactsPage.jsx - bagian fetchMessages()
const decrypted = await Promise.all(
  raw.map(async (msg) => {
    try {
      // Lapisan 1: Verifikasi MAC sebelum dekripsi
      if (msg.mac && hmacKey) {
        const valid = await verifyMAC(hmacKey, msg.ciphertext, msg.mac);
        if (!valid)
          return { ...msg, isMine, macFailed: true, plaintext: null };
      }
      // Lapisan 2: Dekripsi AES-256-GCM
      const plaintext = await aesDecrypt(aesKey, msg.ciphertext, msg.iv);
      return { ...msg, isMine, plaintext, decryptionFailed: false };
    } catch {
      // Tangkap error dekripsi - tandai pesan sebagai gagal
      return { ...msg, isMine, plaintext: null, decryptionFailed: true };
    }
  })
);
```

4.7 Implementasi MAC

File relevan: client/src/crypto/mac.js

Implementasi MAC menggunakan algoritma HMAC-SHA-256 melalui Web Crypto API. Material kunci HMAC diturunkan dari *shared secret* ECDH menggunakan HKDF dengan nilai `info = "chat-hmac-key-v1"`, terpisah dari kunci AES yang menggunakan `info = "chat-aes-key-v1"`, sehingga kedua kunci bersifat independen secara kriptografis meskipun berasal dari material yang sama.

Terdapat dua fungsi utama. Fungsi `computeMAC()` menghitung MAC dari ciphertext menggunakan `crypto.subtle.sign()` dan mengembalikan hasilnya dalam format base64. MAC dihitung dari ciphertext (bukan plaintext) sehingga verifikasi dapat dilakukan sebelum proses dekripsi. Fungsi `verifyMAC()` memverifikasi MAC menggunakan `crypto.subtle.verify()` yang bekerja secara *constant-time* sehingga aman terhadap *timing attack*. Jika format input tidak valid, fungsi menangkap *exception* dan mengembalikan `false`.

```
// client/src/crypto/mac.js
export async function computeMAC(hmacKey, ciphertext) {
  const data = base64ToBuffer(ciphertext);
  const macBuffer = await crypto.subtle.sign("HMAC", hmacKey, data);
  return bufferToBase64(macBuffer);
}

export async function verifyMAC(hmacKey, ciphertext, mac) {
  try {
    const data = base64ToBuffer(ciphertext);
    const macBuffer = base64ToBuffer(mac);
    return await crypto.subtle.verify("HMAC", hmacKey, macBuffer, data);
  } catch {
    return false;
  }
}
```

4.8 Implementasi unit test JWT

File relevan:

- server/src/tests/jwt.sign.test.js
- server/src/tests/jwt.verify.test.js

Unit test diimplementasikan menggunakan framework **Jest** yang dikonfigurasi pada server/package.json. Key pair ECDSA untuk pengujian di-generate menggunakan Node.js crypto dan disimpan pada direktori server/src/tests/jwt.keys/ untuk ketiga algoritma (ES256, ES384, ES512).

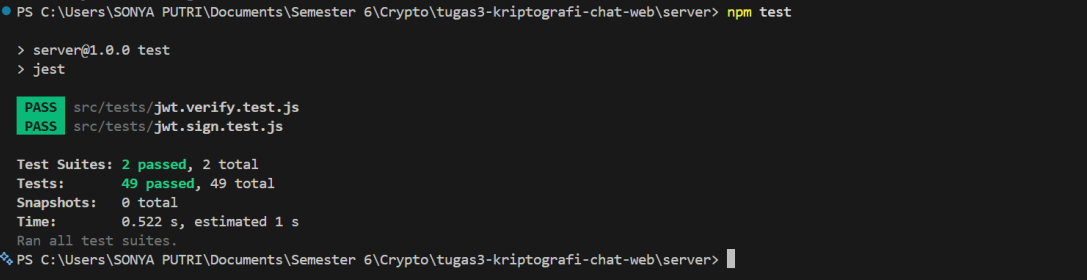
Terdapat dua file test dengan total **49 test case**:

- **jwt.sign.test.js**
9 *happy path* dan 12 *edge case*, mencakup keberhasilan sign untuk ketiga algoritma, validasi format *output*, *encoding header* dan *payload*, berbagai tipe data pada payload, serta penolakan input tidak valid seperti *header* kosong, algoritma tidak didukung, *private key* salah, dan *payload circular reference*.
- **jwt.verify.test.js**
11 *happy path* dan 17 *edge case*, mencakup keberhasilan *verify*, validasi klaim waktu dan identitas, opsi *ignoreExp*/*ignoreNbf*, serta penolakan token dengan signature dimodifikasi, *payload* diubah, format tidak valid, token *expired*, dan *public key* yang tidak sesuai.

```
// server/src/tests/jwt.sign.test.js (cuplikan edge case)
test("[EC-9] Payload berisi circular reference → melempar JWTError", () => {
  const circular = {};
  circular.self = circular;
  expect(() =>
    sign(makeHeader("ES256"), {}, circular, keys.ES256.private)
  ).toThrow(JWTError);
});

// server/src/tests/jwt.verify.test.js (cuplikan edge case)
test("[EC-7] Signature dimodifikasi → melempar JWTError", () => {
  const token = sign(makeHeader("ES256"), {}, { x: 1 },
    keys.ES256.private);
  const parts = token.split(".");
  parts[2] = parts[2].slice(0, -4) + "ZZZZ";
  expect(() => verify(parts.join("."),
    keys.ES256.public)).toThrow(JWTError);
});
```

Seluruh test dijalankan dengan perintah `npm test` dari direktori `server` dan menghasilkan **49 passed, 0 failed** sebagai berikut.



```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web\server> npm test

> server@1.0.0 test
> jest

PASS src/tests/jwt.verify.test.js
PASS src/tests/jwt.sign.test.js

Test Suites: 2 passed, 2 total
Tests: 49 passed, 49 total
Snapshots: 0 total
Time: 0.522 s, estimated 1 s
Ran all test suites.
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web\server>
```

Gambar 4.5 Unit Tes JWT

4.9 Implementasi Docker

File relevan:

- `docker-compose.yml`
- `client/Dockerfile`
- `server/Dockerfile`

`docker-compose.yml` mendefinisikan orchestration untuk tiga service: `db` menggunakan image `postgres:16-alpine` dengan volume `pgdata` dan `healthcheck` untuk memastikan database siap sebelum service lain dijalankan, `server` dibangun dari konteks `server` dengan `environment` untuk koneksi database dan konfigurasi JWT (ES256, 3600 detik), serta `client` dibangun dari konteks `client` dengan `VITE_API_URL` untuk komunikasi ke *backend*. Service `server` bergantung pada kondisi `service_healthy` dari `db`, dan service `client` bergantung pada `server`.

Dockerfile menggunakan base image `node:22-alpine`, menginstal dependensi dengan `npm ci`, menyalin seluruh aplikasi, mengekspos port 5173, dan menjalankan `npm run dev -- --host 0.0.0.0`.

Dockerfile juga menggunakan `node:22-alpine`, menginstal dependensi dengan `npm ci`, menyalin seluruh server, mengekspos port 3000, dan menjalankan `node scripts/migrate.js` diikuti `npm run start` untuk menjalankan migrasi database saat container dimulai.

Dengan konfigurasi ini, seluruh aplikasi (database, backend, dan frontend) dapat dijalankan secara terisolasi dalam container melalui Docker Compose.

4.10 Penjelasan file penting pada repository

Repository ini menggunakan struktur direktori terpisah untuk client dan server. Berikut penjelasan file dan folder penting:

Direktori `client/`

- `src/pages/` berisi halaman utama aplikasi (`LoginPage.jsx`, `RegisterPage.jsx`, `ContactsPage.jsx`, `ChatPage.jsx`)
- `src/components/` berisi komponen UI pendukung seperti `ContactList`, `MessageBubble`, `Sidebar`, dan `ErrorMessage`
- `src/api/` berisi modul API client untuk komunikasi HTTP (`userApi.js`, `messageApi.js`, `authApi.js`)
- `src/crypto/` berisi implementasi algoritma kriptografi: `aes.js` (enkripsi/dekripsi AES-256-GCM), `ecdh.js` (key exchange), `hkdf.js` (key derivation), `mac.js` (HMAC), `passwordKey.js` (password-based key derivation), dan `encoding.js` (utility encoding/decoding)
- `src/storage/authStorage.js` mengelola penyimpanan token JWT dan kunci kriptografi di memori sesi (session storage)
- `src/routes/ProtectedRoute.jsx` implementasi rute yang dilindungi dengan verifikasi JWT
- `vite.config.js` konfigurasi Vite bundler untuk development dan build frontend
- `package.json` daftar dependensi frontend dan script npm (`npm run dev`, `npm run build`, `npm run test:crypto`)

Direktori server/

- `src/app.js` konfigurasi Express app dengan *middleware* (CORS, helmet, autentikasi)
- `src/index.js` *entry point* server yang menjalankan aplikasi pada port yang dikonfigurasi
- `src/config/` berisi konfigurasi environment (`env.js`) dan database (`db.js`)
- `src/database/` berisi `schema.sql` (definisi tabel PostgreSQL) dan `seed.js` (data dummy untuk pengujian)
- `src/jwt-lib/` custom JWT *library* dengan fungsi `sign/verify` (`sign.js`, `verify.js`), utility `base64url` (`base64url.js`), error handler (`errors.js`), dan key loader (`keyLoader.js`)
- `src/routes/` mendefinisikan endpoint API untuk autentikasi, pengguna, dan pesan
- `src/controllers/` mengimplementasikan logika bisnis untuk setiap *endpoint*
- `src/services/` berisi layanan database dan operasi kriptografi di sisi server
- `src/middleware/auth.middleware.js` *middleware* untuk verifikasi JWT pada setiap request yang membutuhkan autentikasi
- `src/utils/` berisi utility functions seperti `passwordHash.js` (bcrypt hashing), `response.js` (formatter respons), dan `validator.js` (validasi input)
- `src/tests/` berisi unit test JWT (`jwt.sign.test.js`, `jwt.verify.test.js`) dan direktori `jwt.keys/` yang menyimpan pasangan kunci ECDSA untuk testing
- `scripts/migrate.js` menjalankan `schema.sql` untuk membuat tabel database, `reset-db.js` membersihkan data pengujian
- `package.json` daftar dependensi server dan script npm (`npm run dev`, `npm start`, `npm test`, `npm run migrate`)

File konfigurasi root

- `docker-compose.yml` mendefinisikan orchestration tiga service (db, server, client) dengan konfigurasi *environment* dan *volume*
- `package.json` root untuk mengelola workspace monorepo jika diperlukan
- `README.md` dokumentasi proyek

Alur kerja penggunaan file

Ketika user melakukan registrasi, data melalui pipeline: client → authApi.js → POST /api/auth/register → auth.controller.js → auth.service.js → database. Sebaliknya, ketika user mengirim pesan, alur enkripsi dimulai dari crypto/ modules di client, pesan dienkripsi, dikirim melalui messageApi.js, disimpan terenkripsi di database, dan penerima melakukan dekripsi menggunakan crypto modules yang sama.

V. Pengujian dan Analisis

5.1 Registrasi dengan data valid

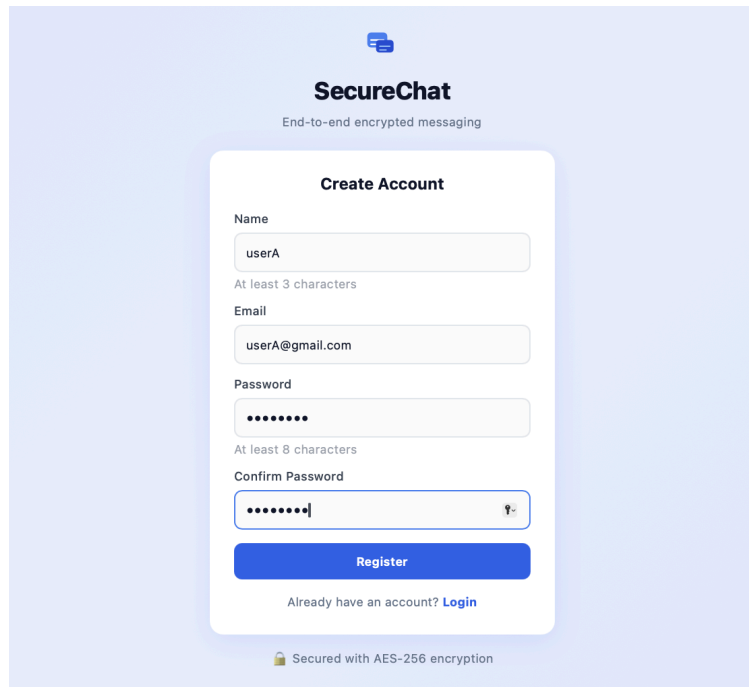
Tujuan: Memverifikasi bahwa sistem dapat menerima data registrasi yang valid dan menyimpan data pengguna dengan benar ke basis data, termasuk memastikan password tidak tersimpan dalam bentuk *plaintext*.

Prosedur pengujian:

1. Pengujian dapat dilakukan dengan 2 cara, yang pertama mengirimkan request POST ke endpoint /api/auth/register dengan data userA sebagai berikut

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "username": "userA",
  "email": "userA@gmail.com",
  "password": "12345678",
  "publicKey": "-----BEGIN PUBLIC
KEY-----\nMFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE...\n-----EN
D PUBLIC KEY-----",
  "encryptedPrivateKey": "base64encodedencryptedkey",
  "iv": "base64iv",
  "salt": "base64salt"
}'
```

2. Cara kedua, registrasi secara langsung pada web



Gambar 5.1 Unit Tes Registrasi Valid - Registrasi melalui Web

3. Setelah request dikirim, dilakukan verifikasi data pada basis data dengan query berikut:

```
psql -U postgres -d webchat -c \  
"SELECT id, username, email, password_hash, password_salt, created_at  
FROM users;"
```

Hasil Pengujian:

1. Server mengembalikan respon dengan status *success*

```
almafelicia@almas-macbook-air-2 server % curl -X POST http://localhost:3000/api/auth/register \  
-H "Content-Type: application/json" \  
-d '{  
  "username": "userA",  
  "email": "userA@gmail.com",  
  "password": "12345678",  
  "publicKey": "-----BEGIN PUBLIC KEY-----\nMFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE...\n-----END PUBLIC KEY-----",  
  "encryptedPrivateKey": "base64encodedencryptedkey",  
  "iv": "base64iv",  
  "salt": "base64salt"  
}',  
{  
  "success": true,  
  "message": "Registration successful",  
  "data": {  
    "user": {  
      "id": 1,  
      "username": "userA",  
      "email": "userA@gmail.com",  
      "created_at": "2026-05-12T05:39:56.734Z"  
    }  
  }  
}
```

Gambar 5.2 Unit Tes Registrasi Valid - Hasil respon server

2. Hasil *query database* menunjukkan data tersimpan sebagai berikut

id	username	email	password_hash	password_salt	created_at
1	userA	userA@gmail.com	\$2b\$12\$H3imBortAGDwoQtMKsY1en9L2wMybZf8xLxPnPy9rhR67wJ7K	4cbe4e66a249f31217a7e28d1b862972e9c2e7b188471d6922f3f74da8219df	2026-05-12 12:19:59.027184
(1 row)					

Gambar 5.3 Unit Tes Registrasi Valid - Hasil query database

Hasil *query* memperlihatkan bahwa password 12345678 tidak tersimpan dalam bentuk plaintext, melainkan sudah dalam bentuk hash bcrypt yang diawali dengan identifier \$2b\$12\$ yang menunjukkan penggunaan bcrypt dengan cost factor 12, dikombinasikan dengan salt unik pada kolom password_salt. *Public key* dan *encrypted private key* juga tersimpan dengan benar sesuai data yang dikirimkan oleh klien. Pengujian ini membuktikan bahwa sistem berhasil memproses registrasi dengan data valid dan mengamankan kredensial pengguna sesuai ketentuan.

5.2 Registrasi dengan email terdaftar

Tujuan: Memverifikasi bahwa sistem menolak pendaftaran akun baru apabila email yang digunakan sudah terdaftar sebelumnya, serta memastikan data pengguna yang sudah ada tidak tertimpa.

Prosedur Pengujian:

1. Registrasi dilakukan kembali menggunakan email userA@gmail.com yang sudah terdaftar pada pengujian 5.1, sebagai berikut:

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "username": "duplicate",
  "email": "userA@gmail.com",
  "password": "12345678",
  "publicKey": "...",
  "encryptedPrivateKey": "...",
  "iv": "...",
  "salt": "..."
}'
```

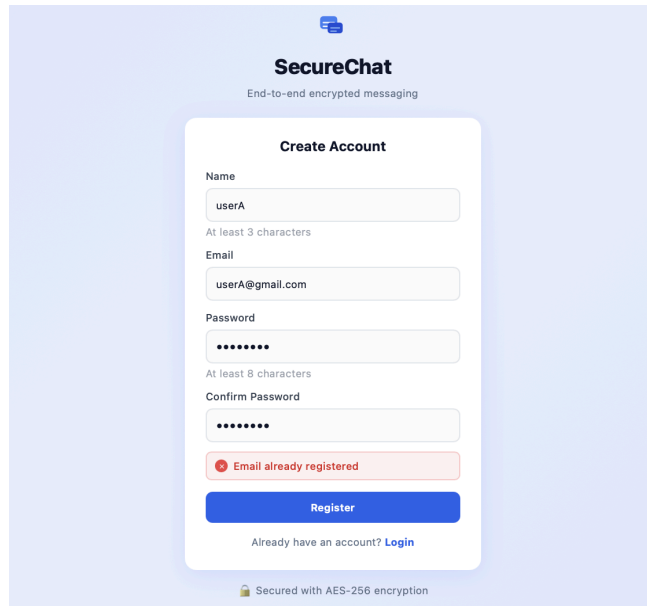
Hasil Pengujian:

1. Server mengembalikan response dengan status fail

```
almafeliccia@almas-macbook-air-2 server % curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "username": "duplicate",
  "email": "userA@gmail.com",
  "password": "12345678",
  "publicKey": "...",
  "encryptedPrivateKey": "...",
  "iv": "...",
  "salt": "..."
}'
{"success":false,"message":"Email already registered"}
```

Gambar 5.4 Unit Tes Registrasi Email terdaftar - Hasil respon server

2. Pengujian langsung pada web akan menghasilkan *notice* “email already registered”



Gambar 5.5 Unit Tes Registrasi Email terddaftar - Respon web

Sistem berhasil mendeteksi bahwa email userA@gmail.com sudah terdaftar dan mengembalikan *success* : false tanpa memproses data registrasi baru. Hasil query pada basis data memperlihatkan bahwa hanya terdapat satu record dengan email tersebut dan data asli userA tidak mengalami perubahan apapun. Pengujian ini membuktikan bahwa *constraint unique* pada kolom email di basis data berfungsi dengan benar dan ditangani oleh sistem dengan *respons error* yang sesuai.

5.3 Login dengan password benar

Tujuan: Memverifikasi bahwa sistem berhasil mengautentikasi pengguna dan menerbitkan JWT yang valid ketika email dan *password* yang diberikan sesuai dengan data yang tersimpan di basis data.

Prosedur Pengujian:

1. *Login* dilakukan dengan mengirimkan request POST ke *endpoint* /api/auth/login menggunakan kredensial userA sebagai berikut:

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "userA@gmail.com",
  "password": "12345678"
}'
```



```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "userA@gmail.com",
  "password": "wrongpassword"
}'
```

Hasil Pengujian:

Server mengembalikan *response* dengan *success: false* karena *invalid credentials* sebagai berikut:

```
● almafelicia@almas-macbook-air-2 server % curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "userA@gmail.com",
  "password": "wrongpassword"
}'
{"success":false,"message":"Invalid credentials"}%
```

Gambar 5.7 Unit Tes Login Salah - Hasil respon server

Sistem berhasil mendeteksi ketidaksesuaian *password* dan menolak permintaan autentikasi tanpa menerbitkan JWT. Perlu diperhatikan bahwa pesan error yang dikembalikan bersifat generik dan tidak membedakan antara kondisi email yang tidak terdaftar dengan kondisi password yang salah. Pendekatan ini disengaja untuk mencegah *user enumeration attack*, yaitu teknik di mana penyerang dapat mengetahui apakah suatu email terdaftar dalam sistem hanya berdasarkan perbedaan pesan error yang dikembalikan oleh server.

5.5 JWT sign dan verify dengan public key benar

Tujuan: Memverifikasi bahwa fungsi `sign()` dapat menghasilkan JWT yang valid dan fungsi `verify()` berhasil membaca header serta payload menggunakan *public key* yang benar.

Prosedur Pengujian:

1. Masuk ke direktori server dan jalankan unit test JWT:

```
cd server
npm test -- --runInBand
```

2. Perhatikan test case yang relevan pada `jwt.verify.test.js`, yaitu pengujian *sign* lalu *verify* berhasil untuk ketiga algoritma (ES256, ES384, ES512).

Hasil:

```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> cd server
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web\server> npm test -- --runInBand

> server@1.0.0 test
> node --experimental-vm-modules ./node_modules/jest/bin/jest.js --runInBand

(node:33104) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS src/tests/jwt.verify.test.js
PASS src/tests/jwt.sign.test.js

Test Suites: 2 passed, 2 total
Tests: 49 passed, 49 total
Snapshots: 0 total
Time: 0.816 s, estimated 1 s
Ran all test suites.
```

Gambar 5.8 Unit Tes JWT - JWT sign dan verify dengan public key benar

Berdasarkan hasil pengujian, token JWT berhasil dibuat menggunakan fungsi `sign()` dengan private key ES256 (maupun ES384 dan ES512). Token kemudian diverifikasi menggunakan fungsi `verify()` dengan public key yang sesuai, dan *payload* berhasil dibaca dengan benar. Hal ini membuktikan bahwa implementasi custom JWT library telah sesuai dengan standar RFC 7519 dan RFC 7515 untuk alur *sign* dan *verify* normal.

5.6 JWT verify dengan public key salah

Tujuan: Memverifikasi bahwa fungsi `verify()` menolak token yang diverifikasi menggunakan *public key* yang bukan pasangan dari private key pembuat token.

Prosedur Pengujian:

1. Masuk ke direktori server dan jalankan unit test JWT:

```
cd server
npm test -- --runInBand
```

2. Perhatikan test case yang relevan pada `jwt.verify.test.js`, yaitu pengujian token ES256 yang diverifikasi menggunakan *public key* dari *key pair* lain (ES384).

Hasil:

```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> cd server
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web\server> npm test -- --runInBand

> server@1.0.0 test
> node --experimental-vm-modules ./node_modules/jest/bin/jest.js --runInBand

(node:33104) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS src/tests/jwt.verify.test.js
PASS src/tests/jwt.sign.test.js

Test Suites: 2 passed, 2 total
Tests: 49 passed, 49 total
Snapshots: 0 total
Time: 0.816 s, estimated 1 s
Ran all test suites.
```

Gambar 5.9 Unit Tes JWT - JWT verify dengan public key salah

Berdasarkan hasil pengujian, token yang dibuat menggunakan *private key* ES256 tidak dapat diverifikasi menggunakan *public key* dari *key pair* yang berbeda. Karena *public key* tidak sesuai dengan *private key* pembuat token, proses verifikasi *signature* gagal dan fungsi `verify()` melempar `JWTError`. Hal ini membuktikan bahwa sistem tidak dapat dipalsukan atau diverifikasi menggunakan kunci yang salah, sehingga integritas token terjamin.

5.7 JWT format tidak valid

Tujuan: Memverifikasi bahwa fungsi `verify()` menolak token yang formatnya tidak sesuai dengan standar JWT, yaitu tiga segmen yang dipisahkan oleh titik (`header.payload.signature`).

Prosedur Pengujian:

1. Masuk ke direktori server dan jalankan unit test JWT:

```
cd server
npm test -- --runInBand
```

2. Perhatikan test case yang relevan pada `jwt.verify.test.js`, yaitu pengujian token dengan jumlah segmen yang tidak valid, mencakup token dengan satu segmen, dua segmen, dan empat segmen.

Hasil:

```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> cd server
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web\server> npm test -- --runInBand

> server@1.0.0 test
> node --experimental-vm-modules ./node_modules/jest/bin/jest.js --runInBand

(node:33104) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS src/tests/jwt.verify.test.js
PASS src/tests/jwt.sign.test.js

Test Suites: 2 passed, 2 total
Tests: 49 passed, 49 total
Snapshots: 0 total
Time: 0.816 s, estimated 1 s
Ran all test suites.
```

Gambar 5.10 Unit Tes JWT - JWT format tidak valid

Berdasarkan hasil pengujian, token yang tidak memiliki tepat tiga segmen berhasil dideteksi sebagai tidak valid oleh fungsi `verify()`. Seluruh variasi format yang tidak sesuai, baik token dengan satu, dua, maupun empat segmen, masing-masing menyebabkan fungsi `verify()` melempar `JWTError`. Hal ini membuktikan bahwa sistem telah mengimplementasikan validasi format token sesuai standar RFC 7519.

5.8 JWT exp/nbf tidak sesuai

Tujuan: Memverifikasi bahwa fungsi `verify()` menolak token yang telah kedaluwarsa (exp sudah terlewat) maupun token yang belum berlaku (nbf di masa depan).

Prosedur Pengujian:

1. Masuk ke direktori server dan jalankan unit test JWT:

```
cd server
npm test -- --runInBand
```

2. Perhatikan test case yang relevan pada `jwt.verify.test.js`, mencakup dua skenario:
 - Token dengan klaim `exp` yang sudah terlewat dari waktu saat ini.
 - Token dengan klaim `nbf` yang berada di masa depan.

Hasil:

```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> cd server
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web\server> npm test -- --runInBand

> server@1.0.0 test
> node --experimental-vm-modules ./node_modules/jest/bin/jest.js --runInBand

(node:33104) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS src/tests/jwt.verify.test.js
PASS src/tests/jwt.sign.test.js

Test Suites: 2 passed, 2 total
Tests: 49 passed, 49 total
Snapshots: 0 total
Time: 0.816 s, estimated 1 s
Ran all test suites.
```

Gambar 5.11 Unit Tes JWT - JWT exp/nbf tidak sesuai

Berdasarkan hasil pengujian, kedua skenario klaim waktu berhasil divalidasi dengan benar oleh fungsi `verify()`. Token dengan nilai `exp` yang sudah terlewat ditolak karena dianggap kadaluarsa, sedangkan token dengan nilai `nbf` di masa depan ditolak karena belum memasuki waktu berlakunya. Pada kedua kasus tersebut, fungsi `verify()` melempar `JWTError` dengan pesan yang sesuai. Hal ini membuktikan bahwa implementasi validasi klaim waktu telah berjalan sesuai ketentuan RFC 7519.

5.9 ECDH menghasilkan shared secret sama

Tujuan: Memverifikasi bahwa dua pengguna yang melakukan *key exchange* menggunakan ECDH dapat menghasilkan *shared secret* yang identik di kedua sisi tanpa perlu saling mengirimkan kunci rahasia.

Prosedur Pengujian:

1. Masuk ke direktori client dan jalankan crypto self-test:

```
cd client
npm run test:crypto
```

2. Perhatikan output *shared secret* yang dihasilkan oleh Alice dan Bob pada terminal.

Hasil:

```
> cipherchat-client@1.0.0 test:crypto
> node src/crypto/crypto.selftest.mjs

Alice Shared Secret: nS7+LoB72k3gSTmG3BG2RK00vbvtkr80FFCc+6J20CY=
Bob Shared Secret:   nS7+LoB72k3gSTmG3BG2RK00vbvtkr80FFCc+6J20CY=
```

Gambar 5.12 Alice dan Bob Shared Secret pada output npm run test:crypto

Berdasarkan hasil pengujian, Alice dan Bob masing-masing menghitung *shared secret* menggunakan *private key* miliknya sendiri dan *public key* milik pihak lawan. Nilai *shared secret* yang dihasilkan oleh kedua sisi terbukti identik. Hal ini membuktikan bahwa protokol ECDH dengan kurva P-256 berhasil diimplementasikan dengan benar melalui Web Crypto API, di mana kedua pihak dapat menghasilkan *secret* yang sama tanpa server mengetahui nilainya.

5.10 HKDF menghasilkan kunci simetris

Tujuan: Memverifikasi bahwa *shared secret* hasil ECDH berhasil diturunkan menjadi kunci AES-256 (256 bit / 32 byte) yang valid menggunakan HKDF.

Prosedur Pengujian:

1. Buka terminal
2. Jalankan perintah berikut untuk menjalankan crypto self-test:

```
cd client
npm run test:crypto
```

Hasil:

```
Alice AES Key: f64c81afdc889e45644081157367ca3d4cb2ce64fdac0d2ddbc56a834b73b45e
Bob AES Key:   f64c81afdc889e45644081157367ca3d4cb2ce64fdac0d2ddbc56a834b73b45e
```

Gambar 5.13 Kunci AES-256 yang dihasilkan HKDF di sisi User A dan B

Berdasarkan hasil pengujian, nilai AES-256 *key* hasil derivasi HKDF pada sisi User A dan User B identik. Hal ini membuktikan bahwa kedua pengguna berhasil menghasilkan *shared secret* ECDH yang sama dan *shared secret* tersebut berhasil diturunkan menjadi symmetric key AES-256 yang valid menggunakan HKDF.

5.11 Pesan berhasil dienkripsi dan didekripsi

Tujuan: Memverifikasi bahwa pesan yang dikirim User A tampil dalam bentuk ciphertext di server atau basis data, namun dapat dibaca dengan benar oleh User B dalam bentuk plaintext.

Prosedur Pengujian:

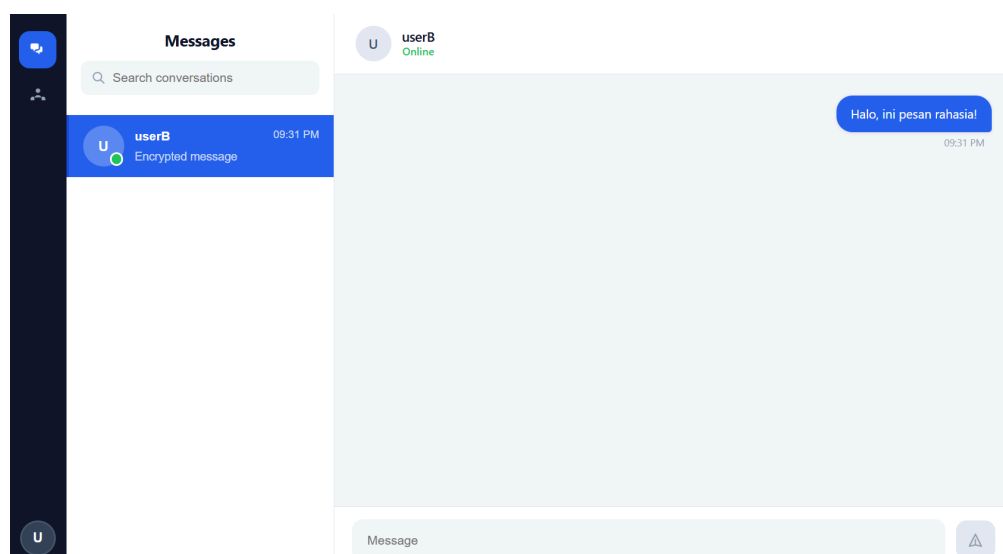
1. Buka dua tab browser. Tab 1 login sebagai akun A dan Tab 2 login sebagai akun B.
2. Pada User A, buka percakapan dengan User B dan kirimkan sebuah pesan plaintext (misalnya: "Halo, ini pesan rahasia!").
3. Buka terminal, lalu masuk ke PostgreSQL dan periksa data yang tersimpan untuk memastikan pesan tersimpan dalam bentuk ciphertext (bukan plaintext).

```
psql -h localhost -p 5432 -U postgres -d webchat  
Lalu masukkan password:
```

```
SELECT id, sender_email, receiver_email, ciphertext, iv, mac, timestamp  
FROM messages  
ORDER BY timestamp DESC;
```

4. Pada User B buka percakapan dengan User A.

Hasil:



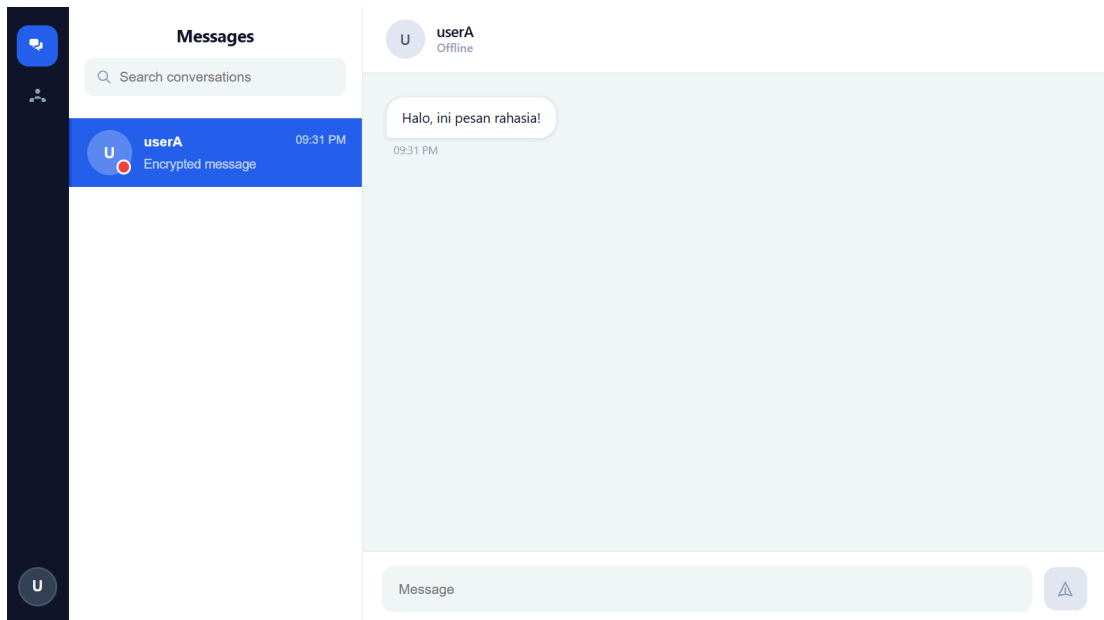
Gambar 5.14 Tampilan pesan terkirim di sisi User A

```

webchat=# SELECT id, sender_email, receiver_email, ciphertext, iv, mac, timestamp
webchat=# FROM messages
webchat=# ORDER BY timestamp DESC;
id | sender_email | receiver_email | ciphertext | iv | mac |
-----+-----+-----+-----+-----+-----+
1 | usera@gmail.com | userb@gmail.com | TaxuSuVPzF8wsAEcj5aguVcf2PgdkwXCqYji5HmK+YbxQMDSVfvtfQ== | cxThFN6xurr/F0ns | z2gx13wt+c9sjploi0wa+N49f19JNS3NSbJJz |
(1 row)

```

Gambar 5.15 Data pesan terenkripsi (ciphertext) yang tersimpan di basis data/server



Gambar 5.16 Tampilan pesan di sisi User B (plaintext terdekripsi dengan benar)

Berdasarkan hasil pengujian, pesan yang dikirim oleh User A berhasil dienkripsi sebelum dikirim ke server. Hal ini terlihat dari data pada basis data yang tersimpan dalam bentuk ciphertext beserta IV dan MAC, sehingga isi pesan asli tidak dapat dibaca secara langsung dari server atau database.

Ketika User B membuka percakapan, pesan berhasil didekripsi menggunakan kunci simetris yang telah dibentuk melalui proses ECDH dan HKDF, sehingga pesan dapat ditampilkan kembali dalam bentuk *plaintext* yang sesuai dengan pesan asli yang dikirim oleh User A.

5.12 Dekripsi gagal dengan kunci/data salah

Tujuan: Memverifikasi bahwa sistem menampilkan penanda *error* ketika proses dekripsi gagal akibat penggunaan kunci yang salah atau data ciphertext yang telah dimodifikasi.

Prosedur Pengujian:

1. *Login* menggunakan dua akun berbeda pada dua tab browser terpisah (User A dan User B).
2. Kirim pesan dari User A ke User B melalui aplikasi chat.
3. Buka terminal, lalu masuk ke PostgreSQL dan ambil data pesan terakhir menggunakan *query* berikut:

```
SELECT id, ciphertext, iv, mac
FROM messages
ORDER BY timestamp DESC
LIMIT 1;
```

4. Ubah nilai ciphertext pada pesan tersebut agar data terenkripsi menjadi rusak, misalnya dengan *query*:

```
UPDATE messages
SET ciphertext = 'AAAA'
WHERE id = <ID_PESAN>;
```

5. Tunggu proses *polling* aplikasi atau *refresh* halaman chat pada User B dan periksa tampilan pesan pada antarmuka aplikasi.

Hasil:

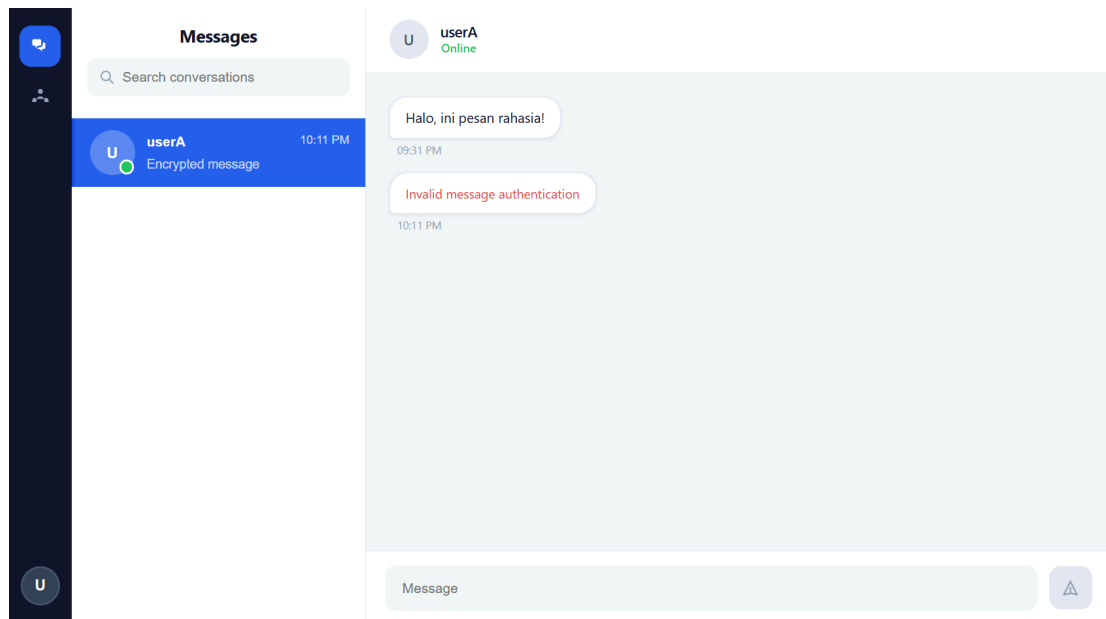
```
webchat=# SELECT id, ciphertext, iv, mac
webchat=# FROM messages
webchat=# ORDER BY timestamp DESC
webchat=# LIMIT 1;
 id | ciphertext | iv | mac
-----+-----+-----+-----
  2 | VHY28vtfUDyVy3xFpJMTfJWmgQjspyQ= | 2yb1SYZ5mT8Z+ccU | BRggf31TLdVxa1yrvKz420Frk9vJoxv4d2RNw50c00o=
(1 row)
```

Gambar 5.17 Ciphertext pesan sebelum dimodifikasi pada basis data

```
webchat=# UPDATE messages
webchat=# SET ciphertext = 'AAAA'
webchat=# WHERE id = 2;
UPDATE 1

webchat=# SELECT id, ciphertext, iv, mac
webchat=# FROM messages
webchat=# ORDER BY timestamp DESC
webchat=# LIMIT 1;
 id | ciphertext | iv | mac
-----+-----+-----+-----
  2 | AAAA | 2yb1SYZ5mT8Z+ccU | BRggf31TLdVxa1yrvKz420Frk9vJoxv4d2RNw50c00o=
(1 row)
```

Gambar 5.18 Ciphertext pesan setelah dimodifikasi pada basis data



Gambar 5.19 Tampilan error akibat ciphertext yang dimodifikasi

Berdasarkan hasil pengujian, ciphertext pesan yang tersimpan pada basis data berhasil dimodifikasi secara langsung melalui PostgreSQL. Setelah ciphertext diubah menjadi data yang tidak valid, aplikasi tidak dapat melakukan proses dekripsi pesan pada sisi User B.

Hal ini ditunjukkan dengan munculnya pesan *error* saat percakapan dimuat kembali. Pengujian ini membuktikan bahwa sistem mampu mendeteksi ciphertext yang telah dirusak atau dimanipulasi sehingga plaintext asli tidak dapat dipulihkan kembali.

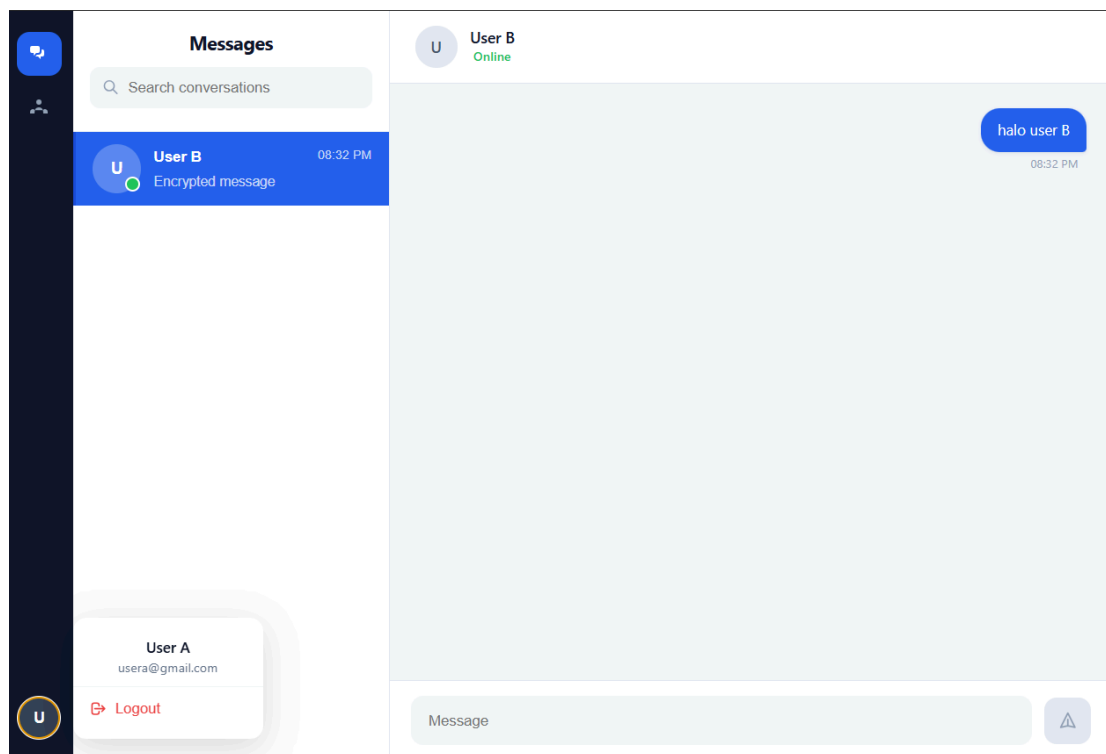
5.13 MAC valid

Tujuan: Memverifikasi bahwa pesan yang dikirim dengan MAC yang valid dapat diterima dan ditampilkan dalam bentuk plaintext oleh penerima.

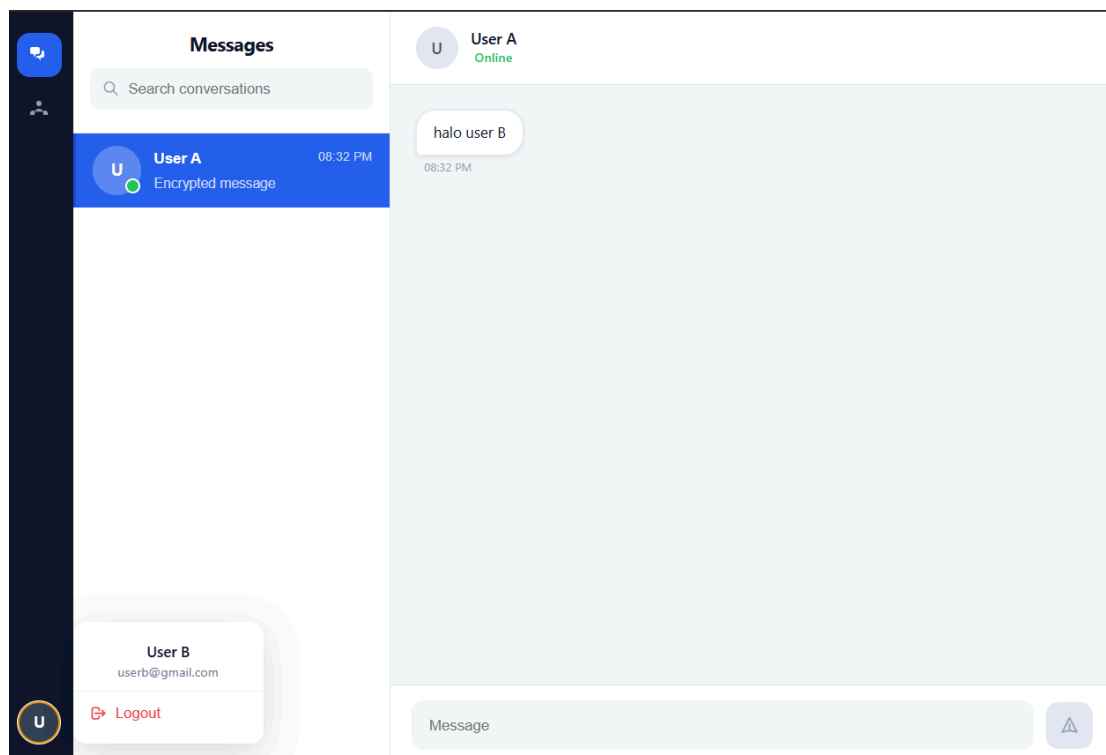
Prosedur Pengujian:

1. Buka dua tab browser. Tab 1 *login* sebagai User A dan Tab 2 *login* sebagai User B.
2. Pada User A, kirimkan pesan baru ke User B (misalnya: "halo user B").
3. Tanpa melakukan perubahan apapun pada basis data, buka percakapan pada tab User B dan periksa tampilan pesan.

Hasil:



Gambar 5.20 Tampilan pesan terkirim di sisi User A



Gambar 5.21 Tampilan pesan diterima dalam bentuk *plaintext* di sisi User B

Berdasarkan hasil pengujian, pesan yang dikirim oleh User A berhasil diterima dan ditampilkan dalam bentuk *plaintext* oleh User B. Hal ini menunjukkan bahwa MAC yang dihitung menggunakan HMAC-SHA-256 pada sisi pengirim berhasil diverifikasi oleh sisi penerima, sehingga pesan dinyatakan valid dan proses dekripsi AES-256-GCM dapat dilanjutkan. Pengujian ini membuktikan bahwa mekanisme MAC berjalan dengan benar untuk memastikan integritas dan autentisitas pesan.

5.14 MAC tidak valid setelah pesan diubah

Tujuan: Memverifikasi bahwa sistem menolak pesan dan menampilkan penanda *error* ketika nilai MAC tidak sesuai akibat pesan atau MAC yang telah dimanipulasi.

Prosedur Pengujian:

1. Buka dua tab browser. Tab 1 *login* sebagai User A dan Tab 2 *login* sebagai User B.
2. Pada User A, kirimkan pesan baru ke User B.
3. Buka terminal, masuk ke PostgreSQL dan ambil data pesan terakhir:

```
SELECT id, ciphertext, iv, mac
FROM messages
ORDER BY timestamp DESC
LIMIT 1;
```

4. Ubah nilai MAC pada pesan tersebut menjadi nilai yang tidak valid:

```
UPDATE messages
SET mac = 'invalid-mac'
WHERE id = <ID_PESAN>;
```

5. Tunggu proses *polling* aplikasi sekitar 2 detik atau *refresh* halaman chat pada tab User B, lalu periksa tampilan pesan pada antarmuka aplikasi.

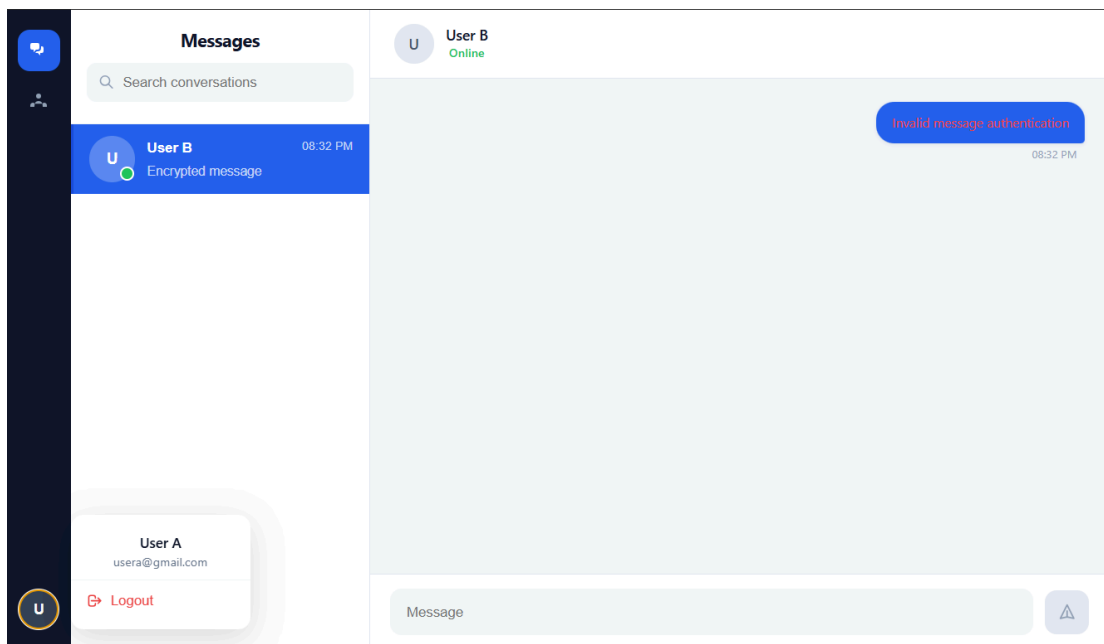
Hasil:

```
webchat=# SELECT id, ciphertext, iv, mac
webchat=# FROM messages
webchat=# ORDER BY timestamp DESC
webchat=# LIMIT 1;
 id |          ciphertext          |          iv          |          mac
-----+-----+-----+-----
  1 | fi5vgrSjGs8MwymPs4r7Ap8BrNeTho+9suu/ | j4nVh7kNG/VX5bbg | 5RZ5Y1AcgQFo7GULh10AHDPPeVX
6FbTp+n1qf+ayDFY=
(1 row)
```

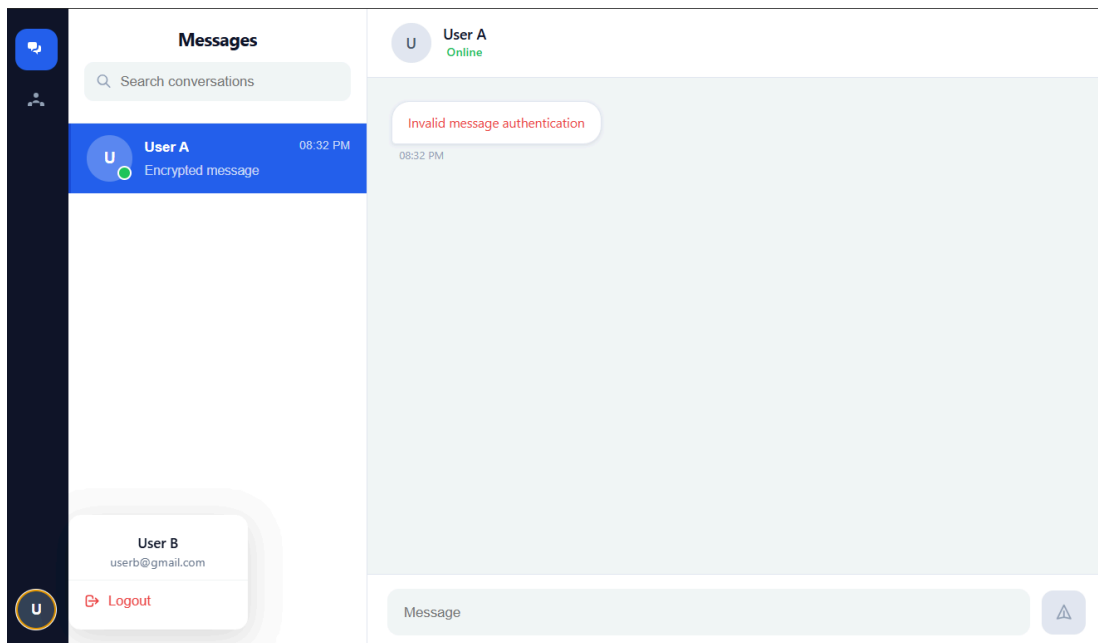
Gambar 5.22 Data MAC sebelum dimodifikasi pada basis data

```
webchat=# SELECT id, mac
webchat=# FROM messages
webchat=# WHERE id = 1;
 id | mac
-----+-----
  1 | invalid-mac
(1 row)
```

Gambar 5.23 Data MAC setelah dimodifikasi menjadi 'invalid-mac' pada basis data



Gambar 5.24 Tampilan "Invalid message authentication" di sisi User A



Gambar 5.25 Tampilan "*Invalid message authentication*" di sisi User B

Berdasarkan hasil pengujian, setelah nilai MAC pada basis data diubah menjadi nilai yang tidak valid, sistem berhasil mendeteksi ketidaksesuaian tersebut. Pesan tidak didekripsi dan antarmuka aplikasi menampilkan pesan *error* "Invalid message authentication" pada kedua sisi. Hal ini membuktikan bahwa mekanisme verifikasi MAC menggunakan `crypto.subtle.verify()` berjalan dengan benar, sehingga pesan yang telah dimanipulasi dapat terdeteksi dan ditolak sebelum proses dekripsi dilakukan.

5.15 Pengujian Docker

Tujuan: Memverifikasi bahwa seluruh komponen aplikasi, yaitu client, server, dan database, dapat dijalankan secara konsisten menggunakan Docker Compose dalam satu perintah.

Prosedur Pengujian:

1. Pastikan Docker Desktop telah berjalan, lalu jalankan perintah berikut untuk membangun dan menjalankan seluruh *container*:

```
docker compose up --build
```

2. Tunggu hingga seluruh *service* selesai dibangun dan berjalan. Pastikan output terminal menunjukkan ketiga service (db, server, client) telah aktif.

3. Verifikasi koneksi database dengan masuk ke *container* PostgreSQL:

```
docker compose exec db psql -U postgres -d webchat
```

4. Periksa data yang tersimpan pada basis data untuk memastikan aplikasi berjalan dengan benar:

```
SELECT * FROM users;  
SELECT id, sender_email, receiver_email, ciphertext, iv, mac FROM  
messages;
```

5. Setelah selesai, hentikan seluruh *container*:

```
docker compose down -v
```

Hasil:

```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> docker compose up --build  
[+] Building 4.8s (20/20) FINISHED  
=> [internal] load local bake definitions 0.0s  
=> => reading from stdin 1.26kB 0.0s  
=> [client internal] load build definition from Dockerfile 0.0s  
=> => transferring dockerfile: 184B 0.0s  
=> [server internal] load build definition from Dockerfile 0.0s  
=> => transferring dockerfile: 192B 0.0s  
=> [client internal] load metadata for docker.io/library/node:22-alpine 2.5s  
=> [client internal] load .dockerignore 0.0s  
=> => transferring context: 2B 0.0s  
=> [server internal] load .dockerignore 0.1s  
=> => transferring context: 2B 0.0s  
=> [client 1/5] FROM docker.io/library/node:22-alpine@sha256:8ea2348b068a9544dae7317b4f3aafcdc032df16  
=> => resolve docker.io/library/node:22-alpine@sha256:8ea2348b068a9544dae7317b4f3aafcdc032df1647bb7d7 0.0s  
=> [client internal] load build context 0.5s  
=> => transferring context: 324.94kB 0.5s  
=> [server internal] load build context 0.9s  
=> => transferring context: 563.65kB 0.9s  
=> CACHED [server 2/5] WORKDIR /app 0.0s  
=> CACHED [client 3/5] COPY package*.json ./ 0.0s  
=> CACHED [client 4/5] RUN npm ci 0.0s  
=> CACHED [client 5/5] COPY . . 0.0s  
=> [client] exporting to image 0.2s  
=> => exporting layers 0.0s  
=> => exporting manifest sha256:dbc7d1878a983d890ee3a055451c719ea1107802df37e233675e034f3d028272 0.0s  
=> => exporting config sha256:d5e23a137184ed93b0a3c946e3bf5f64ab26d18bee9da72345559c957f539605 0.0s  
=> => exporting attestation manifest sha256:bb0b2190b2f31494183daf6949acb29d740871c58ed49278bc10aff61 0.0s  
=> => exporting manifest list sha256:26bc215fba4b197dfb49dee3e4a72a4911a7f6a4bba521b643b4a86e1342765a 0.0s  
=> => naming to docker.io/library/tugas3-kriptografi-chat-web-client:latest 0.0s  
=> [client] resolving provenance for metadata file 0.0s  
=> CACHED [server 3/5] COPY package*.json ./ 0.0s  
=> CACHED [server 4/5] RUN npm ci 0.0s  
=> CACHED [server 5/5] COPY . . 0.0s  
=> [server] exporting to image 0.2s  
=> => exporting layers 0.0s  
=> => exporting manifest sha256:8cde98f036d8ae585224a180397bdc4042edc9c98f12edacc342d54bc41c51 0.0s
```

Gambar 5.26 Output terminal proses build Docker Compose menunjukkan seluruh layer berhasil dibangun

```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> docker compose up --build
[+] Building 4.8s (20/20) FINISHED

server-1 |
client-1 |

server-1 | < injected env (0) from .env // tip: % enable debugging { debug: true }
client-1 | 2:23:59 PM [vite] (client) Re-optimizing dependencies because vite config has changed

server-1 | Connected to PostgreSQL

client-1 |

client-1 | VITE v6.4.2 ready in 375 ms
server-1 | Server running on port 3000

client-1 |
server-1 | Environment: development

client-1 | → Local: http://localhost:5173/
client-1 | → Network: http://172.19.0.4:5173/
█

View in Docker Desktop View Config Enable Watch
```

Gambar 5.27 Output terminal menunjukkan ketiga service (db, server, client)

berjalan dengan server terhubung ke PostgreSQL dan client berjalan pada port 5173

```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> docker compose exec db psql -U postgres -d webchat
psql (16.13)
Type "help" for help.

webchat=# SELECT * FROM users;
 id | username | email | password_hash | public_key | password_salt |
-----+-----+-----+-----+-----+-----+
 1 | Nia | sonya@gmail.com | $2b$12$1T84ZtkVczhMoCpWngHwJ.GE3Mm2T5oAiHbriQLwXJc0H1R2z5The | 50e751376718510a0a08d702e625c47d90d07ae8e22b9fb2906a6cf9e8df81fb | MFkwEwYHkoZTizj0CAQYIKoZIj0DAQcDQgAEu0ZoybnLPxANuNgPbvKOGRP0rJrpe+kqv7L3nImPU1mTKAi5xyV6YpkGyv4oom#Ma+FW0ss9syXlpE0/wqLw== | 3sX/THGcmkoMg0WazFmbOck6Qgrlw5jKKsLEv0I9geYIdJzu1G9K1KPPE5iPyy76eu/vElm0w1lWmEQj17YSD0uIeJvdYrFL+VwXOZZxrp7GHUKLh0ckRFxvhBZ5UhWqYr-fon3Key2LM+J6HYZ2P2gCadC8r-fn14pr2BnwquXPd68/Yuch1WgYIGSr7CUvkpm4bqxmqkan4w== | YayWjASv5ZLIsc/a | ENJR20F+hGvYb+7Rmz9eGA== | t |
 2026-05-05 14:19:48.386724 | 2026-05-05 14:06:52.596639
 2 | aul | aulia@gmail.com | $2b$12$52FmyhRd0DSvgznZQn951.Xt9n8Gg4tsyobWpPQa0Xc09Xy1ZxMPm | 562f471cdb33775587727df4c04bc193026c4c21934daef0747420e3a63ab2bed | MFkwEwYHkoZTizj0CAQYIKoZIj0DAQcDQgAEKWK1/dsJjP5j8zDRGTesD4e4IvNoyeywtbX0/a/EResZK/VzJTeJZhExd5E2UtlZXVNAZg9rk3yE2Nb1R1FA== | 0XpgcadCPw6Ppf1QwVtuA2Wq5D5wj/nsGT/NeQzbEuG4PfoTyPrqRD+HPZEWh5kUCs/FjiHMB609Jo/gMKgcESLYC5z+AH/5+Lbgq6i/RonEFQDzfi7xdzgmSQyYNY74SKOF0EOzmbdTu6nLUZ9bNuhDCF/JCRUQIr-JGdKxL48KBrKZP1AnKLX57mr4L03YTDXq5ZCFqY4saA== | OqvQ20udtkMkGv7 | QBesYBjKvGQ3sJucgSbaLw== | f |
 2026-05-05 14:19:08.778069 | 2026-05-05 14:18:32.799326
(2 rows)

webchat=# SELECT id, sender_email, receiver_email, ciphertext, iv, mac FROM messages;
 id | sender_email | receiver_email | ciphertext | iv | mac |
-----+-----+-----+-----+-----+-----+
 1 | aulia@gmail.com | sonya@gmail.com | CafuXxFA9qQuNiUdaZMEzk7ofME= | uhg/BmohopVda+NV | EAFUu9I1/thMcP+770zK0irrMkRSGpDJBVQjd1lw+8= |
 2 | sonya@gmail.com | aulia@gmail.com | 4FlF10Aj5151adK8M7IwST+PQ= | 8PBmUkBAU2cm9up4 | w0PDRvSW00PqMxj2IenBpMerbuV9Ah7v+wwWtpqZU= |
(2 rows)

webchat=#
```

Gambar 5.28 Data users dan messages yang tersimpan pada PostgreSQL di dalam container

```
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> docker compose down -v
[+] Running 5/5
✓ Container tugas3-kriptografi-chat-web-client-1 Removed 0.5s
✓ Container tugas3-kriptografi-chat-web-server-1 Removed 1.1s
✓ Container tugas3-kriptografi-chat-web-db-1 Remov... 0.4s
✓ Network tugas3-kriptografi-chat-web_default Rem... 0.3s
✓ Volume tugas3-kriptografi-chat-web_pgdata Remov... 0.0s
PS C:\Users\SONYA PUTRI\Documents\Semester 6\Crypto\tugas3-kriptografi-chat-web> 
```

Gambar 5.29 Output terminal docker compose down -v menunjukkan seluruh *container* berhasil dihentikan

Berdasarkan hasil pengujian, seluruh komponen aplikasi berhasil dijalankan menggunakan satu perintah `docker compose up --build`. *Service* db (PostgreSQL) berhasil dijalankan terlebih dahulu, diikuti service server (Node.js/Express) yang terhubung ke database, dan *service client* (React/Vite) yang berjalan pada port 5173. Data pengguna dan pesan terenkripsi terbukti tersimpan dengan benar pada basis data di dalam *container*. Pengujian ini membuktikan bahwa konfigurasi Docker Compose telah berjalan dengan baik dan aplikasi dapat dijalankan secara konsisten di berbagai lingkungan.

VI. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, aplikasi *chat* berbasis web yang dikembangkan telah berhasil memenuhi seluruh kebutuhan utama pada tugas ini. Sistem mampu mengimplementasikan mekanisme autentikasi menggunakan JSON Web Token (JWT) berbasis JSON Web Signature (JWS) dengan algoritma ECDSA melalui library yang dibangun secara mandiri tanpa menggunakan *library* JWT eksternal.

Selain itu, aplikasi berhasil menerapkan mekanisme pembentukan kunci komunikasi menggunakan *Elliptic Curve Diffie-Hellman* (ECDH) yang dipadukan dengan HKDF untuk menghasilkan kunci simetris AES-256-GCM. Dengan pendekatan ini, proses enkripsi dan dekripsi pesan dilakukan sepenuhnya di sisi klien sehingga server hanya berperan sebagai perantara penyimpanan dan pengiriman data terenkripsi tanpa mengetahui isi plaintext pesan maupun shared secret yang digunakan antar pengguna.

Pengujian yang dilakukan menunjukkan bahwa sistem mampu menangani berbagai skenario dengan baik, mulai dari registrasi dan *login* pengguna, validasi JWT, proses *key exchange*, enkripsi dan dekripsi pesan, hingga verifikasi integritas pesan menggunakan HMAC. Sistem juga mampu mendeteksi kondisi kesalahan seperti JWT tidak valid, MAC yang dimodifikasi, maupun ciphertext yang rusak.

Selain memenuhi requirement utama, aplikasi juga berhasil mengimplementasikan fitur bonus berupa mekanisme *Message Authentication Code* (MAC), unit test untuk *library* JWT, serta *deployment* menggunakan Docker dan Docker Compose. Dengan demikian, aplikasi yang dikembangkan tidak hanya memenuhi aspek fungsionalitas, tetapi juga menerapkan prinsip keamanan komunikasi modern berbasis kriptografi kunci publik dan kunci simetris secara terintegrasi pada platform web.

DAFTAR PUSTAKA

- [1] Krawczyk, H., & Eronen, P. (2010). HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869. IETF. <https://www.rfc-editor.org/rfc/rfc5869>

- [2] Barker, E., Chen, L., Roginsky, A., Vassilev, A., & Davis, R. (2018). Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography. NIST SP 800-56A Rev. 3. <https://doi.org/10.6028/NIST.SP.800-56Ar3>

- [3] National Institute of Standards and Technology. (2001). Advanced Encryption Standard (AES). FIPS PUB 197. <https://doi.org/10.6028/NIST.FIPS.197>

- [4] D. McGrew dan J. Viega, "The Galois/Counter Mode of Operation (GCM)," Submission to NIST Modes of Operation Process, 2004 dan Dworkin, M. (2007). Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST SP 800-38D. <https://csrc.nist.gov/publications/detail/sp/800-38d/final>

- [5] Mozilla Developer Network, "Web Crypto API," MDN Web Docs. [Online]. Tersedia: https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API

LAMPIRAN

A. Pranala repository

<https://github.com/sonyaaputri/tugas3-kriptografi-chat-web>

B. Pranala video demo

 18223112_18223131_18223138_Video Demo 3_II4021.mp4

C. Pembagian tugas

NIM	Nama	Tugas	
18223112	Alma Felicia Vielrizki	Kode	server/src/app.js server/src/index.js server/src/config/ server/src/database/ server/src/routes/ server/src/controllers/ server/src/services/ server/src/middleware/ server/src/Utils/passwordHash.js
		Laporan	1. Pendahuluan 2.6 Password hashing dan salt 3.1 Arsitektur client-server 3.2 Skema basis data 3.3 Alur registrasi 3.4 Alur login 4.3 Implementasi backend API 4.5 Implementasi database 5.1 Registrasi valid 5.2 Registrasi duplikat 5.3 Login benar 5.4 Login salah
18223131	Aulia Azka Azzahra	Kode	client/src/pages/ client/src/components/ client/src/api/ client/src/storage/

			client/src/crypto/aes.js client/src/crypto/encoding.js client/src/styles/main.css
		Laporan	2.5 AES-256 2.4 HKDF 3.5 Alur daftar kontak 3.7 Alur pengiriman pesan terenkripsi + Membuat diagram alur 3.5 - 3.9 4.1 Implementasi frontend 4.2 Implementasi API client 4.6 Implementasi enkripsi dan dekripsi pesan 5.10 HKDF menghasilkan kunci simetris 5.11 Pesan berhasil dienkripsi dan didekripsi 5.12 Dekripsi gagal
18223138	Sonya Putri Fadilah	Kode	client/src/crypto/ecdh.js client/src/crypto/hkdf.js client/src/crypto/passwordKey.js client/src/crypto/mac.js server/src/jwt-lib/ server/src/tests/
		Laporan	2.1 JWT 2.2 JWS dan ECDSA 2.3 ECDH 2.7 MAC 2.8 Docker 3.6 Alur key exchange 3.8 Alur MAC 3.9 Docker 4.4 Implementasi JWT 4.7 Implementasi MAC 4.8 Unit test JWT 4.9 Docker 4.10 Penjelasan File Penting pada Repository 5.5-5.8 JWT testing 5.9 ECDH shared secret

			5.13 MAC valid 5.14 MAC tidak valid 5.15 Docker test 6 Kesimpulan
--	--	--	--